

BATCH No:29

**BLOCKVERIFY: A RELIABLE
CREDENTIALING APPLICATION**

*Major project report submitted
in partial fulfillment of the requirement for award of the degree of*

**Bachelor of Technology
in
Information Technology**

By

U. VARSHITHA (21UTIT0057)

*Under the guidance of
Mrs. LIJETHA C JAFFRIN, B.TECH., M.E.,
ASSISTANT PROFESSOR*



**DEPARTMENT OF INFORMATION TECHNOLOGY
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF
SCIENCE AND TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

**Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2025

BATCH No:29

**BLOCKVERIFY: A RELIABLE
CREDENTIALING APPLICATION**

*Major project report submitted
in partial fulfillment of the requirement for award of the degree of*

**Bachelor of Technology
in
Information Technology**

By

U. VARSHITHA (21UTIT0057)

*Under the guidance of
Mrs. LIJETHA C JAFFRIN, B.TECH., M.E.,
ASSISTANT PROFESSOR*



**DEPARTMENT OF INFORMATION TECHNOLOGY
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF
SCIENCE AND TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

**Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2025

CERTIFICATE

It is certified that the work contained in the project report titled "BLOCKVERIFY: A RELIABLE CREDENTIALING APPLICATION" by U. VARSHITHA (21UTIT0057) has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor

Mrs. Lijetha C Jaffrin

Assistant Professor

Information Technology

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

May, 2025

Signature of Head of the Department

Dr. Visumathi. J.

Professor & Head

Information Technology School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

May, 2025

Signature of the Dean

Dr. S P. Chokkalingam

Professor & Dean

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

May, 2025

DECLARATION

We declare that this written submission represents my ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

(U.VARSHITHA)

Date: / /

APPROVAL SHEET

This project report entitled BLOCKVERIFY: A RELIABLE CREDENTIALING APPLICATION by U.VARSHITHA (21UTIT0057) is approved for the degree of B.Tech in Information Technology.

Examiners

Supervisor

Mrs. LIJETHA C JAFFRIN, B.TECH., M.E.

Assistant Professor,.

Date: / /

Place:

ACKNOWLEDGEMENT

We express our deepest gratitude to our **Honorable Founder Chancellor and President Col. Prof. Dr. R. RANGARAJAN B.E. (Electrical), B.E. (Mechanical), M.S (Automobile), D.Sc., and Foundress President Dr. R. SAGUNTHALA RANGARAJAN M.B.B.S., Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for her blessings.**

We express our sincere thanks to our respected Chairperson and Managing Trustee **Dr. Mrs. RANGARAJAN MAHALAKSHMI KISHORE, B.E., Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for their blessings.**

We are very much grateful to our beloved **Vice Chancellor Prof. Dr. RAJAT GUPTA**, for providing us with an environment to complete our project successfully.

We record indebtedness to our **Professor & Dean, School of Computing, Dr. S P. CHOKKALINGAM, M.Tech., Ph.D., & Professor & Associate Dean, School of Computing, Dr. V. DHILIP KUMAR, M.E., Ph.D.,** for immense care and encouragement towards us throughout the course of this project.

We are thankful to our **Professor & Head, Department of Information Technology, Dr. Visumathi. J., M.E., Ph.D.,** for providing immense support in all our endeavors.

We also take this opportunity to express a deep sense of gratitude to our **Internal Supervisor Mrs. LIJETHA C JAFFRIN, B.TECH., M.E.,** for her cordial support, valuable information and guidance, she helped us in completing this project through various stages.

A special thanks to our **Project Coordinators Dr. ANITHA RAJAKUMARI. P., M.E., Ph.D.,** for their valuable guidance and support throughout the course of the project.

We thank our department faculty, supporting staff and friends for their help and guidance to complete this project.

U.VARSHITHA (21UTIT0057)

ABSTRACT

Credential verification plays a critical role in validating the authenticity of certifications across academic, professional, and organizational domains; however, traditional systems often fall short due to their centralized nature, making them prone to fraud, data manipulation, inefficiencies, and costly administrative overhead. These legacy systems typically lack transparency and are vulnerable to security breaches, as they rely heavily on manual verification processes and centralized authorities. In contrast, blockchain technology offers a decentralized, immutable, and secure framework for credential verification, effectively addressing these limitations. By leveraging smart contracts and cryptographic mechanisms, blockchain enables transparent, tamper-proof recordkeeping, thereby eliminating risks associated with data loss and unauthorized alterations. The integration of decentralized verification systems facilitates real-time validation, reduces dependency on intermediaries, and minimizes administrative efforts, ultimately enhancing trust and security in the certification process. This study explores the transformative potential of blockchain-based credential verification systems, analyzing how they can bolster digital security, ensure data integrity, and streamline verification processes. Furthermore, it investigates the practical implementation of this approach using a technology stack comprising MongoDB for scalable data storage, Next.js for the user interface, Node.js for backend operations, and Solidity for automating credential issuance and validation through smart contracts. While highlighting the benefits, the study also delves into the challenges of adopting blockchain in large-scale credentialing infrastructures, such as integration complexities, user adoption, and regulatory considerations. Ultimately, blockchain-based credential verification represents a paradigm shift in digital identity management, promising enhanced transparency, security, and efficiency in the way credentials are issued and authenticated.

Keywords: Blockchain, Credential Verification, Decentralization, Data Integrity, Digital Identity Management, Digital Trust, Smart Contracts, Transparency.

LIST OF FIGURES

Figure No.	Title of Figure	Page No.
4.1	Architecture Diagram of Certificate Generation using Blockchain	17
4.2	Data Flow Diagram of Certificate Generation using Blockchain .	18
4.3	Use Case Diagram of Certificate Generation using Blockchain .	19
4.4	Class Diagram of Certificate Generation using Blockchain . . .	20
4.5	Sequence Diagram of Certificate Generation using Blockchain .	21
4.6	Collaboration Diagram of Certificate Generation using Blockchain	22
4.7	Activity Diagram of Certificate Generation using Blockchain . .	23
4.8	String Storage	26
4.9	Smart Contract Deployment	28
4.10	Certificate Verification	29
5.1	Input Design of Certificate Generation using Blockchain	32
5.2	Output Design of Certificate Generation using Blockchain . . .	33
5.3	Unit Testing Test Result of Certificate Generation using Blockchain	35
5.4	Integration Testing Test Result of Certificate Generation using Blockchain	37
5.5	System Testing Test Result of Certificate Generation using Blockchain	39
5.6	Performance Evaluation	40
6.1	Comparative Analysis Graph	44
10.1	Plagiarism Report of Certificate generation using blockchain . .	53
11.1	Poster Presentation	61

LIST OF TABLES

Table No.	Title of Table	Page No.
6.1	Comparison of Existing and Proposed System	43

LIST OF ACRONYMS AND ABBREVIATIONS

S.NO	ACRONYMS	ABBREVIATION
1.	BC	Blockchain
2.	CVS	Credential Verification System
3.	EVM	Ethereum Virtual Machine
4.	ETH	Ethereum
5.	DApp	Decentralized Application

TABLE OF CONTENTS

	Page.No
ABSTRACT	v
LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ACRONYMS AND ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Background	2
1.3 Objective	3
1.4 Problem Statement	4
2 LITERATURE REVIEW	5
2.1 Existing System	7
2.2 Related Work	7
2.3 Research Gap	8
3 PROJECT DESCRIPTION	9
3.1 Existing System	9
3.2 Proposed System	10
3.3 Feasibility Study	11
3.3.1 Economic Feasibility	11
3.3.2 Technical Feasibility	12
3.3.3 Social Feasibility	12
3.4 System Specification	13
3.4.1 Hardware Specification	13
3.4.2 Software Specification	14
3.4.3 Tools and Technologies Used	15
3.4.4 Standards and Policies	16

4	SYSTEM DESIGN AND METHODOLOGY	17
4.1	System Architecture	17
4.2	Design Phase	18
4.2.1	Data Flow Diagram	18
4.2.2	Use Case Diagram	19
4.2.3	Class Diagram	20
4.2.4	Sequence Diagram	21
4.2.5	Collaboration diagram	22
4.2.6	Activity Diagram	23
4.3	Algorithm & Pseudo Code	24
4.3.1	Blockchain-Enabled Certificate Issuance and Verification Algorithm	24
4.3.2	Pseudo Code	24
4.4	Module Description	25
4.4.1	String Storage	25
4.4.2	Smart Contract Deployment	26
4.4.3	Certificate Verification	28
4.5	Steps to execute/run/implement the project	29
4.5.1	Set Up Development Environment	29
4.5.2	Create and compile a New Solidity File	30
4.5.3	Deploy the Contract and Interact with the Contract	30
5	IMPLEMENTATION AND TESTING	32
5.1	Input and Output	32
5.1.1	Input Design	32
5.1.2	Output Design	33
5.2	Testing	34
5.2.1	Unit Testing	34
5.2.2	Integration Testing	35
5.2.3	System Testing	37
5.2.4	Performance Evaluation	40
6	RESULTS AND DISCUSSIONS	41
6.1	Efficiency of the Proposed System	41
6.2	Comparison of Existing and Proposed System	42
6.3	Comparative Analysis-Table	43

6.4	Comparative Analysis-Graphical Representation and Discussion . .	44
7	INDUSTRY DETAILS	45
7.1	MegaViz Technologies Pvt Ltd	45
7.1.1	Duration of Internship (27-12-2024 - 2-05-2025)	45
7.1.2	Duration of Internship in month (4)	45
7.1.3	Industry Address : Kukatpally, Hyderabad, Telangana	45
7.2	Internship offer letter	45
7.3	Internship Completion certificate	46
8	CONCLUSION AND FUTURE ENHANCEMENTS	47
8.1	Summary	47
8.2	Limitations	48
8.3	Future Enhancements	48
9	SUSTAINABLE DEVELOPMENT GOALS (SDGs)	50
9.1	Alignment with SDGs	50
9.2	Relevance of the Project to Specific SDG	51
9.3	Potential Social and Environmental Impact	51
10	PLAGIARISM REPORT	53
11	SOURCE CODE	54
11.1	Source Code	54
11.2	Poster Presentation	61
	References	61

Chapter 1

INTRODUCTION

1.1 Introduction

Verifying professional and academic credentials is becoming more difficult in today's digital environment. Conventional systems are ineffective, vulnerable to fraud, and involve a great deal of administrative work since they rely on centralized institutions, such as employers and colleges, to issue and validate certificates. Security and dependability issues are brought up by document manipulation, credential forgery, and sluggish verification procedures. A decentralized, unchangeable, and impenetrable answer to these problems is provided by blockchain technology. A blockchain-based credentialing tool called BlockVerify was created to revolutionize the issuing, storage, and verification of digital certificates. BlockVerify guarantees a safe, open, and effective credentialing system by utilizing blockchain's decentralized structure. In an increasingly digital world, this creative method makes academic and professional credentials more dependable by lowering fraud, boosting confidence, and expediting the verification process.

There are many difficulties with the conventional method of credential issuance and verification. Centralized databases kept by certification bodies and educational institutions are susceptible to illegal changes, data loss, and hacking. Employers and other parties who want to confirm credentials frequently use laborious, manual procedures that include speaking with the granting organizations directly. In addition to raising operating expenses, this also raises the possibility of errors and delays. A viable substitute is offered by the development of blockchain technology, which offers a distributed ledger system that guarantees immutability, security, and transparency. BlockVerify enables direct and trustless verification by storing credential information on a blockchain network, doing away with the need for middlemen.

Modern web technologies, such as Next.js for frontend development, Node.js

for backend processing, MongoDB for database administration, and TypeScript for enhanced code dependability, are used in the creation of BlockVerify. Fundamentally, the application automates credential issuance and verification with Solidity-written Ethereum-based smart contracts. The certification process can be governed by predetermined norms and circumstances thanks to smart contracts, which guarantee that a credential cannot be changed or falsified once it has been issued. Because each certificate is hashed and saved on the blockchain, employers, academic institutions, and other stakeholders may quickly verify it using a safe and intuitive interface.

BlockVerify's capacity to increase confidence in digital credentials while lowering administrative constraints is one of its main benefits. Manual cross-checking, third-party verification services, and copious documentation are all necessary for traditional credentialing systems, and they can be expensive and time-consuming. By facilitating immediate verification via the decentralized ledger of blockchain technology, BlockVerify streamlines this procedure. Without having to get in touch with the issuing authority, employers and institutions can instantly confirm a credential's authenticity, boosting productivity and lowering the risk of fraud. Academic degrees, professional certifications, training program completions, and business credentials are just a few of the many use cases that BlockVerify supports.

It is anticipated that the use of blockchain technology in digital credentialing will grow as it develops further. Implementing decentralized credential verification solutions, such as BlockVerify, can greatly enhance data security, lower fraud, and expedite verification procedures in a variety of industries. Although issues like scalability, data privacy, and regulatory compliance must be resolved, the advantages of blockchain based credentialing systems greatly exceed the drawbacks. BlockVerify sets the standard for safe and effective verification procedures by offering a dependable and transparent solution, ushering in a new era of digital credentialing.

1.2 Background

In an increasingly digital world, the need for secure and tamper-proof certification systems is more critical than ever. Traditional methods of issuing and verifying certificates are often vulnerable to forgery, loss, and inefficiency. Institutions and

employers alike face growing challenges in verifying the authenticity of credentials, leading to a trust gap in many sectors.

BlockVerify addresses this problem by leveraging blockchain technology to create a decentralized platform for certificate issuance and verification. Using smart contracts built on Ethereum, certificates are issued immutably and transparently, ensuring that credentials cannot be altered or duplicated once created. The platform integrates Solidity for smart contract development, Web3.js for blockchain interactions, and Next.js for a seamless front-end user experience.

By decentralizing trust and removing intermediaries, BlockVerify enhances the integrity, accessibility, and verification speed of certificates across education, professional training, and regulatory compliance domains.

1.3 Objective

The scope of this project encompasses the development and deployment of a comprehensive blockchain-based credential verification system that addresses the growing need for secure, transparent, and efficient verification of academic and professional certifications. This system is designed to mitigate the vulnerabilities and inefficiencies of traditional credentialing methods, which are often prone to fraud, manual errors, data breaches, and delays due to their reliance on centralized authorities and physical documentation. The project focuses on implementing a decentralized platform that utilizes the immutability and transparency of blockchain technology, enabling instant verification of credentials without the need for intermediaries. Through the use of Solidity smart contracts on a blockchain network, the system will facilitate the issuance, storage, and validation of credentials in a secure and automated manner. The backend infrastructure will be built using Node.js to handle server-side operations, while the frontend interface will be developed using Next.js to ensure a seamless user experience. MongoDB will serve as the non-relational database to manage user profiles and system metadata, providing scalability and high performance. The system will support multiple stakeholders, including educational institutions, employers, and credential holders, allowing them to interact with the platform for issuing, viewing, and verifying certifications in real-time. This project also includes the integration of cryptographic mechanisms for enhanced data security and privacy, as well as access control features to ensure only authorized parties can issue or validate credentials. The scope further extends to testing the system's

functionality, performance, and resilience against potential threats, as well as evaluating its adoption potential and regulatory considerations for broader implementation across various sectors. Ultimately, this project aims to redefine digital credential verification by creating a tamper-proof, efficient, and trustworthy system that can be adopted globally to support secure identity and qualification validation.

1.4 Problem Statement

Credential verification is a critical component in the academic, professional, and organizational sectors, serving as proof of an individual's qualifications and competencies. However, traditional credentialing systems are primarily centralized, relying on manual processes and third-party authorities to issue and verify certificates. These systems are often time-consuming, costly, and prone to human error, which significantly delays important processes such as job recruitment, admissions, and licensing. Moreover, in an increasingly digital and global environment, the lack of real-time verification capabilities and interoperability among different institutions creates further inefficiencies and bottlenecks in verifying the authenticity of credentials across borders.

Another major issue is the increasing prevalence of forged or altered documents. Centralized databases can be vulnerable to hacking, data tampering, or unauthorized access, which puts the legitimacy of credentials at risk. The absence of tamper-proof mechanisms and transparent audit trails in current systems makes it difficult for employers and institutions to distinguish between genuine and fraudulent credentials. As a result, trust in digital documents is eroding, and verifying credentials often requires direct communication with issuing bodies, which is neither scalable nor efficient.

To overcome these challenges, a more secure, automated, and transparent solution is needed. Blockchain technology, with its decentralized, immutable, and cryptographically secure architecture, offers a strong alternative to traditional verification systems. This project aims to develop a blockchain-based credential verification platform using Solidity smart contracts, supported by MongoDB, Node.js, and Next.js. The goal is to enable secure issuance and real-time verification of credentials, reduce reliance on intermediaries, prevent fraud, and enhance trust in academic and professional records—ultimately streamlining the credential validation process and setting a new standard for digital trust.

Chapter 2

LITERATURE REVIEW

[8] M. Sharples et al. (2016) investigated blockchain's potential for open learning records. In order to provide people complete ownership over their academic and professional credentials and lessen reliance on centralized institutions, their research explores how blockchain can record lifetime learning accomplishments. Their work also emphasizes the benefits of decentralization in supporting continuous and informal learning, allowing learners to build a verifiable portfolio of achievements that can be accessed and validated globally without institutional barriers.

[1] A. Grech et al. (2017) examined several blockchain implementations in academic credentialing as part of a systematic review on the topic. According to the report, blockchain has the ability to end diploma fabrication, facilitate cross-border verification, and guarantee safe peer to-peer credential validation free from middlemen.

[6] J. Alex Halderman et al. (2018) examined the security features of credential verification systems based on blockchain technology. In order to improve system integrity and stop fraudulent activity, the paper suggests cryptographic solutions while assessing possible risks like double issuance and smart contract vulnerabilities. The study highlights the necessity of implementing formal verification techniques and secure smart contract patterns to prevent exploitation. It also stresses the importance of auditability, recommending regular security assessments and immutable logs to detect anomalies in credential issuance.

[9] S. Kim et al. (2019) examined how well blockchain technology works for decentralized identity management. Their research demonstrates how self-sovereign identification (SSI) frameworks and smart contracts can expedite the issuing and verification of digital credentials while maintaining data security and privacy.

[10] T. Nguyen et al. (2020) investigated the use of blockchain technology for digital diploma issuance in higher education. The study presents case studies of colleges that used blockchain-based credentialing systems, showing increases in data transparency, verification speed, and administrative workload reduction. These implementations led to enhanced institutional credibility and student satisfaction. The

study also identified practical challenges such as resistance to change, the need for staff training, and technical integration issues, offering recommendations for smoother adoption of blockchain in academia.

[2] B. K. Mohanty et al. (2021) provided a comparison of different blockchain-based certification systems. In order to optimize blockchain-based solutions, their research assesses various consensus processes, scalability problems, and interoperability hurdles in academic credential verification. The paper discusses how different blockchain platforms (e.g., Ethereum, Hyperledger, and Corda) cater to distinct use cases. They conclude that hybrid models—combining permissioned and public blockchains—can provide the right balance of transparency, control, and scalability for large-scale educational networks.

[3] C. Rong et al. (2021) centered on blockchain-based credentialing applications and smart contract security. In order to improve security in decentralized verification systems, the paper addresses weaknesses such as reentrancy attacks and illegal access and suggests mitigation strategies.

[4] D. Patel et al. (2022) examined how to secure academic records on blockchain networks using cryptographic hashing. Their study offers a thorough examination of hash-based proof techniques, guaranteeing data integrity and immutability in the storage and verification of credentials. They explored techniques like Merkle tree structures and blockchain anchoring to optimize performance. Their findings suggest that hash-based credential models offer lightweight, scalable alternatives to fully on-chain storage without compromising the trust and transparency needed for verification.

[5] E. Liu et al. (2022) carried out a case study on the use of blockchain-based credential verification in enterprises. The study evaluates the influence of blockchain on expediting employment procedures through instant credential validation, implementation problems, and regulatory issues.

[7] K. Singh et al. (2023) explored new developments like zero-knowledge proofs (ZKPs) for privacy-preserving verification as they relate to the future of blockchain in credential verification. Their analysis draws attention to ongoing studies and future advancements in decentralized systems. They demonstrate how ZKPs can validate the authenticity of credentials without revealing underlying data, preserving both user privacy and verification accuracy. This approach is especially promising for applications in privacy-sensitive environments like healthcare, finance, and academic records.

2.1 Existing System

Traditional credential verification systems are centralized and institution-driven, relying heavily on manual processes and third-party validation. Educational institutions issue physical or digital certificates, which recipients must then present to employers or other institutions for verification. These credentials are often stored in centralized databases or printed on paper, making them vulnerable to loss, damage, forgery, and unauthorized modifications. The process of verifying these documents typically involves contacting the issuing authority, which can be time-consuming and inefficient, especially across borders or in large-scale scenarios.

Most institutions use email or web portals for credential sharing and validation, but these systems lack interoperability and standardization. Employers or agencies often face delays and administrative burdens when verifying credentials due to differing formats, internal approval workflows, and limited access to institutional databases. In addition, credentials can be tampered with using software, making it difficult to ensure authenticity without extensive manual checks. These systems also pose risks to data security and privacy, as sensitive information is often stored in unsecured or outdated systems vulnerable to breaches.

Another major limitation is the lack of real-time verification capabilities and self-ownership of credentials. Learners and professionals have minimal control over their certificates and often need to request re-issuance or validation repeatedly. There is also no unified global registry for credential validation, which causes complications when individuals seek opportunities internationally. Overall, the current systems are inefficient, insecure, and lack transparency, which creates a need for a decentralized, tamper-proof, and user-controlled solution—something blockchain technology can address effectively.

2.2 Related Work

Traditional certificate management systems have long relied on centralized servers and institutions to issue, store, and verify academic or professional credentials. These systems often face challenges such as document forgery, unauthorized access, lack of transparency, and difficulty in verification across different platforms. Several digital solutions have been proposed to overcome these issues, utilizing PDF encryption, cloud storage, and secure login systems. However, these methods still

depend heavily on central authorities and are susceptible to data tampering or server failures. Researchers have also explored QR-code-based validation and third-party verification tools, but these methods can be costly, slow, and vulnerable to fraudulent manipulation.

Recent advancements in blockchain technology have sparked significant interest in decentralized solutions for certificate verification. Projects such as MIT's Blockcerts and India's National Academic Depository have showcased how immutable ledgers can be used to securely issue and verify academic records. These systems use smart contracts and distributed networks to ensure data integrity, enhance transparency, and provide lifetime access to credentials. The proposed system in this project builds upon these concepts by implementing Ethereum smart contracts and a user-friendly interface to manage certificate issuance and validation efficiently. Unlike existing models, it emphasizes end-to-end security, real-time verification, and decentralized trust—offering a scalable solution for institutions and learners alike.

2.3 Research Gap

While several existing systems have attempted to digitize certificate issuance and verification, most rely on centralized infrastructures that are prone to data breaches, manipulation, and single points of failure. Current methods often lack transparency and do not provide tamper-proof mechanisms for verifying the authenticity of issued certificates. Furthermore, interoperability among institutions and ease of access for employers or third-party verifiers remains a challenge in traditional systems.

Although blockchain-based frameworks such as Blockcerts have been introduced, they often require complex setup processes, have limited customization options, or are not optimized for widespread adoption in educational institutions with limited technical resources. There is a clear gap in developing a lightweight, secure, and scalable blockchain-powered system that enables real-time certificate issuance and verification, while also being cost-effective, user-friendly, and easily deployable on public networks like Ethereum. This project aims to bridge this gap by providing a practical and robust solution using smart contracts and web-based interfaces tailored for institutional and user needs.

Chapter 3

PROJECT DESCRIPTION

3.1 Existing System

Traditional credential verification systems are centralized and institution-driven, relying heavily on manual processes and third-party validation. Educational institutions issue physical or digital certificates, which recipients must then present to employers or other institutions for verification. These credentials are often stored in centralized databases or printed on paper, making them vulnerable to loss, damage, forgery, and unauthorized modifications. The process of verifying these documents typically involves contacting the issuing authority, which can be time-consuming and inefficient, especially across borders or in large-scale scenarios.

Most institutions use email or web portals for credential sharing and validation, but these systems lack interoperability and standardization. Employers or agencies often face delays and administrative burdens when verifying credentials due to differing formats, internal approval workflows, and limited access to institutional databases. In addition, credentials can be tampered with using software, making it difficult to ensure authenticity without extensive manual checks. These systems also pose risks to data security and privacy, as sensitive information is often stored in unsecured or outdated systems vulnerable to breaches.

Another major limitation is the lack of real-time verification capabilities and self-ownership of credentials. Learners and professionals have minimal control over their certificates and often need to request re-issuance or validation repeatedly. There is also no unified global registry for credential validation, which causes complications when individuals seek opportunities internationally. Overall, the current systems are inefficient, insecure, and lack transparency, which creates a need for a decentralized, tamper-proof, and user-controlled solution—something blockchain technology can address effectively.

3.2 Proposed System

Using blockchain technology, the proposed solution introduces a secure and decentralized mechanism for the issuance and verification of digital certificates. The system is composed of three primary layers: the front-end user interface, the web server, and the blockchain network. This architecture ensures security, transparency, and smooth interaction between users and the blockchain. The system provides an intuitive web-based interface that facilitates certificate issuance and verification. Built using Next.js, the front-end application allows users—including academic institutions, employers, and individuals—to interact with the platform directly through a web browser. Authorized organizations can issue certificates, while third parties can validate them without requiring intermediaries. This interface bridges users and the blockchain network, transmitting data to a Node.js-based server that interacts with the Ethereum Virtual Machine (EVM) to execute smart contract functions.

Once an authorized institution initiates the certificate issuance process, essential data such as the participant's name, course or event name, issuing body, and date are collected. This data is then securely submitted to a Solidity smart contract deployed on the Ethereum blockchain. The smart contract processes and stores the information permanently on the blockchain, generating a unique Certificate ID to ensure authenticity. Rules defined within the contract prevent unauthorized changes and ensure that certificates, once issued, cannot be tampered with. Additionally, the system generates a Block Hash and a Certificate Hash, providing cryptographic assurance of the certificate's integrity and adding another layer of security and verifiability.

All issued certificates are stored in a decentralized manner on the blockchain, making them immutable and easily verifiable. Unlike centralized databases that are prone to manipulation or data loss, blockchain guarantees the permanence and transparency of all stored records. Each certificate is stored as a transaction within a block, safeguarding it against forgery or unauthorized changes. When a third party, such as an employer or university, needs to verify a certificate, they can simply enter the Certificate ID in the verification portal. This request is forwarded by the web application to the blockchain, which retrieves the certificate information from the distributed ledger. Since blockchain data is publicly accessible and unchangeable, the verification process is instantaneous and does not require real-time involvement from the issuing organization. This approach significantly reduces administrative burdens, eliminates fraud, and ensures reliable validation of credentials.

The decentralized architecture of blockchain removes single points of failure, enhancing the system's reliability and security. Even if an issuing organization stops functioning, the certificates it has issued remain verifiable on the blockchain indefinitely. By automating credential management through smart contracts and using a streamlined web interface, the system increases efficiency and minimizes the need for manual validation. Blockchain ensures data protection, while the integration of technologies like Next.js and Node.js offers a responsive and modern user experience. Altogether, this system represents a powerful advancement in secure, trust-based digital credential verification, transforming how academic and professional qualifications are issued and authenticated.

3.3 Feasibility Study

3.3.1 Economic Feasibility

The economic feasibility of the proposed blockchain-based credential verification system is grounded in its capacity to provide a cost-effective alternative to traditional verification methods, while simultaneously enhancing security, operational efficiency, and transparency. Although initial expenditures are required for system development, infrastructure deployment, and personnel training, these are strategically offset by significant long-term advantages. The automation of certificate issuance and verification reduces the reliance on manual processes, thereby lowering administrative overhead and labor costs.

Furthermore, the elimination of physical storage and handling of paper-based certificates minimizes expenses associated with printing, archiving, and document maintenance. The system's robust security mechanisms, including cryptographic validation and immutability, substantially mitigate the risk of credential fraud and unauthorized modifications—thereby avoiding potential financial losses and preserving the integrity and reputation of issuing institutions. In addition, the solution's compatibility with existing digital ecosystems enables seamless integration across various sectors, fostering interoperability, standardization, and further reducing operational expenditures.

In the long term, the system's ability to deliver scalable, secure, and automated credentialing services ensures sustained economic viability. By replacing costly and inefficient traditional systems, it presents a future-ready approach that not only re-

duces financial burden but also establishes trust and reliability in academic and professional verification processes. As digital transformation continues to shape modern institutions, the project stands as a forward-looking investment with strong potential for return and adaptability in a rapidly evolving landscape.

3.3.2 Technical Feasibility

The technical feasibility of the proposed project is grounded in the maturity of blockchain technology and the availability of robust infrastructure required for its successful implementation. By leveraging blockchain, the system ensures key properties such as data integrity, immutability, and decentralization, which form the backbone of a secure and transparent certificate issuance and verification platform. The project benefits from the growing adoption of blockchain frameworks like Ethereum and Hyperledger, alongside readily accessible development tools, libraries, and community support—further enhancing its technical viability.

Security is a core component of the system, achieved through the use of advanced cryptographic techniques that safeguard certificate data from unauthorized access and tampering. The system's ability to integrate with existing credentialing frameworks and legacy infrastructures ensures interoperability, enabling a smooth transition for institutions without major disruptions to their operations. This compatibility encourages broader adoption while preserving continuity for current systems and processes.

Moreover, blockchain's inherent scalability supports the system's growth, allowing it to handle an increasing number of users and certificates without degrading performance or reliability. Overall, the project's technical feasibility is supported by the maturity, flexibility, and security of blockchain technology, combined with its capacity for integration and scalability. These factors make it well-equipped to overcome technical challenges and deliver a sustainable solution for certificate management across educational, organizational, and professional sectors.

3.3.3 Social Feasibility

The social feasibility of the proposed project plays a critical role in its acceptance and widespread adoption among stakeholders, particularly in addressing ongoing concerns surrounding traditional credentialing systems. By leveraging blockchain technology, the project significantly enhances the credibility and trustworthiness of

digital certificates, effectively minimizing the incidence of fraudulent credentials. The system supports inclusivity by offering a transparent and accessible platform for managing certificates, thereby ensuring fair opportunities for individuals across diverse backgrounds. This aligns closely with societal values that emphasize equity, fairness, and equal access to education and professional advancement.

In response to growing public concerns about data privacy and information security, the project places a strong emphasis on protecting user data through advanced cryptographic methods and decentralized storage mechanisms. These features foster user trust and confidence by safeguarding sensitive information from unauthorized access or tampering. Additionally, the automation of verification processes reduces administrative overhead and improves service delivery, fulfilling public expectations for streamlined, efficient, and reliable credential validation.

Overall, the project's ability to address key societal needs—such as transparency, inclusivity, security, and efficiency—positions it as a socially viable solution with the potential for broad acceptance. By modernizing credential management while upholding core social values like fairness, integrity, and trust, the system contributes meaningfully to the advancement of a more secure and equitable digital society.

3.4 System Specification

3.4.1 Hardware Specification

Server Hardware:

- High-performance processors (e.g., Intel Xeon or AMD EPYC)
- Sufficient RAM capacity (e.g., 64GB or higher)
- Enterprise-grade storage solution (e.g., SSDs for faster data access)
- Redundant power supplies for increased reliability

Blockchain Infrastructure:

- Support for Ethereum network requirements
- GPUs for parallel processing (optional for mining or consensus algorithms)
- Solid-State Drives (SSDs) for fast blockchain data retrieval
- Adequate Network Interface Cards (NICs) for efficient communication

User Interface Devices:

- Devices for accessing the Certificate generation(e.g., computers, laptops, or mobile devices)
- Secure input devices (e.g., biometric scanners or secure keypads)
- Robust hardware for durability and reliability in diverse environments

Network Infrastructure:

- High-speed and reliable internet connectivity
- Starlink network for remote areas
- Redundant network architecture to ensure availability
- Firewall and intrusion detection/prevention systems for security

3.4.2 Software Specification**Blockchain Software:**

- Ethereum blockchain protocol
- Solidity smart contract language support
- Compatibility with Ethereum Virtual Machine (EVM)
- Wallet software for secure transaction handling

Smart Contract Development:

- Implementation in Solidity, Ethereum's native programming language
- Integration of OpenZeppelin for secure and standardized contract functionalities

Database Management System:

- Database system compatible with blockchain data storage
- Efficient data retrieval and query capabilities

User Interface Software:

- User-friendly Certificate generation application

- Secure authentication mechanisms (e.g., biometric or multi-factor authentication)
- Accessibility features for diverse user needs

Security Software:

- Antivirus and malware protection
- Firewall and intrusion detection/prevention systems
- Regular security audits and updates

3.4.3 Tools and Technologies Used

Software Stack:

- Next.js – Front-end framework for building dynamic, responsive user interfaces
- MongoDB – NoSQL database for storing user data and metadata
- Node.js – Backend runtime environment to handle server-side logic and API requests
- TypeScript – Superset of JavaScript used for type-safe development and improved maintainability

Blockchain and Smart Contract Tools:

- Solidity – Programming language for writing Ethereum smart contracts
- Web3.js – JavaScript library for interacting with the Ethereum blockchain from the front-end
- MetaMask – Browser extension wallet for managing Ethereum accounts and signing transactions

Development and Collaboration Tools:

- Visual Studio Code (VS Code) – Code editor used for writing, debugging, and managing the codebase
- Git – Version control system to track changes and manage code revisions
- GitHub – Hosting platform for collaboration, code sharing, and project management

3.4.4 Standards and Policies

Information Security Management System (ISMS)

The Information Security Management System (ISMS) of the project is meticulously designed to safeguard the confidentiality, integrity, and availability of certificate data. Through a comprehensive framework of policies, procedures, and technical controls, the ISMS mitigates risks associated with unauthorized access, data breaches, and tampering. Role-based access controls restrict system entry to authorized personnel, while encryption techniques protect data both in storage and during transmission. Regular security audits and penetration testing are conducted to identify and rectify vulnerabilities, ensuring compliance with relevant security standards and regulations. Furthermore, user training and awareness initiatives educate stakeholders on security best practices, fostering a culture of security-conscious behavior. By implementing a robust ISMS, the project aims to instill confidence among users and stakeholders, reaffirming the security and reliability of the credentialing process.

Standard Used: ISO/IEC 27001

Data Privacy and Protection Policy

The Data Privacy and Protection Policy of the project ensures the secure and lawful handling of personal data in accordance with the General Data Protection Regulation (GDPR). It emphasizes data minimization, user consent, and transparency, allowing users to access, modify, or delete their information. Role-based access controls and encryption safeguard data privacy during storage and transmission, while anonymization techniques reduce exposure of personal identifiers. Regular privacy assessments and compliance reviews help maintain alignment with evolving regulations. By embedding privacy into its core processes, the project fosters user trust and ensures responsible data management.

Standard Used: General Data Protection Regulation (GDPR – EU 2016/679)

Chapter 4

SYSTEM DESIGN AND METHODOLOGY

4.1 System Architecture

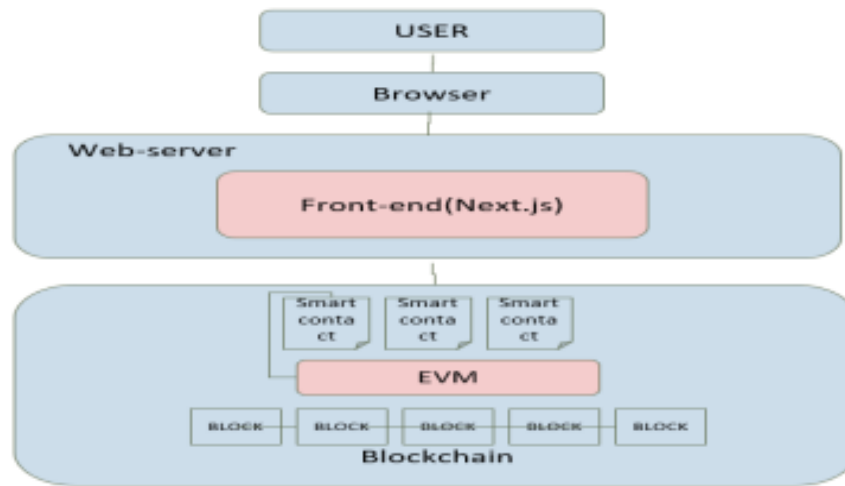


Figure 4.1: Architecture Diagram of Certificate Generation using Blockchain

Figure 4.1 represents a method for issuing and verifying certificates that is based on the Ethereum blockchain and uses its decentralized ledger to store certificates securely. Solidity-based smart contracts guarantee efficiency and dependability by automating issuance and verification. The Next.js-developed user interface offers issuers and verifiers a smooth experience, while Web3.js makes it easier to communicate with the Ethereum Virtual Machine (EVM). Verifiable and tamper proof certifications are guaranteed by the immutable structure of blockchain technology. System resilience is increased by security measures including data integrity checks and smart contract audits. By removing middlemen and boosting trust, this architecture makes it possible for a decentralized, transparent, and fraud-resistant credential management system. The solution solves the shortcomings and inefficiencies of conventional credentialing techniques by utilizing blockchain technology to provide safe, effective, and transparent digital credential verification.

4.2 Design Phase

4.2.1 Data Flow Diagram

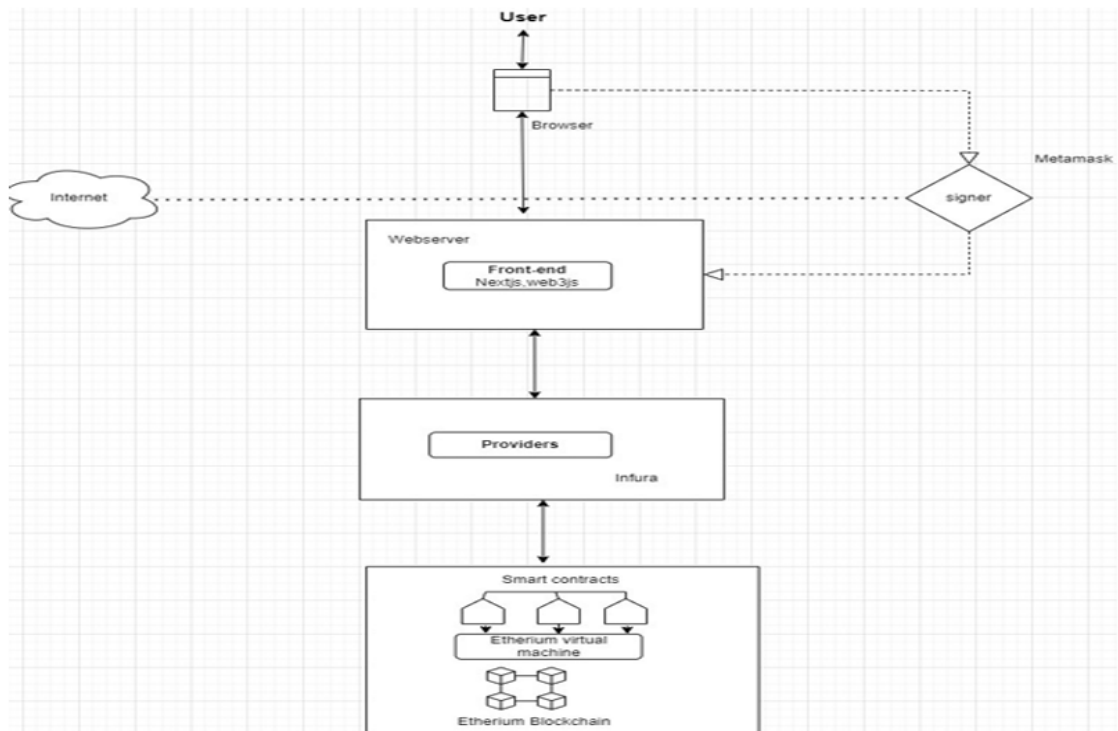


Figure 4.2: Data Flow Diagram of Certificate Generation using Blockchain

Figure 4.2 represents the blockchain network acts as a secure and decentralized ledger, providing a reliable platform for storing and accessing certificate data. User requests for certificate issuance or verification serve as inputs, initiating the flow of data within the system. Processing of these requests occurs through smart contracts, which handle certificate generation and verification efficiently. External components, including user interfaces, wallets, and optional decentralized storage, interact with the system, enriching its functionality. Robust security measures, such as encryption and validation techniques, are implemented to ensure data integrity and confidentiality throughout the process. The data flow within the system illustrates the movement of information between components, facilitating a clear understanding of the certificate processes and enhancing overall transparency and reliability.

4.2.2 Use Case Diagram

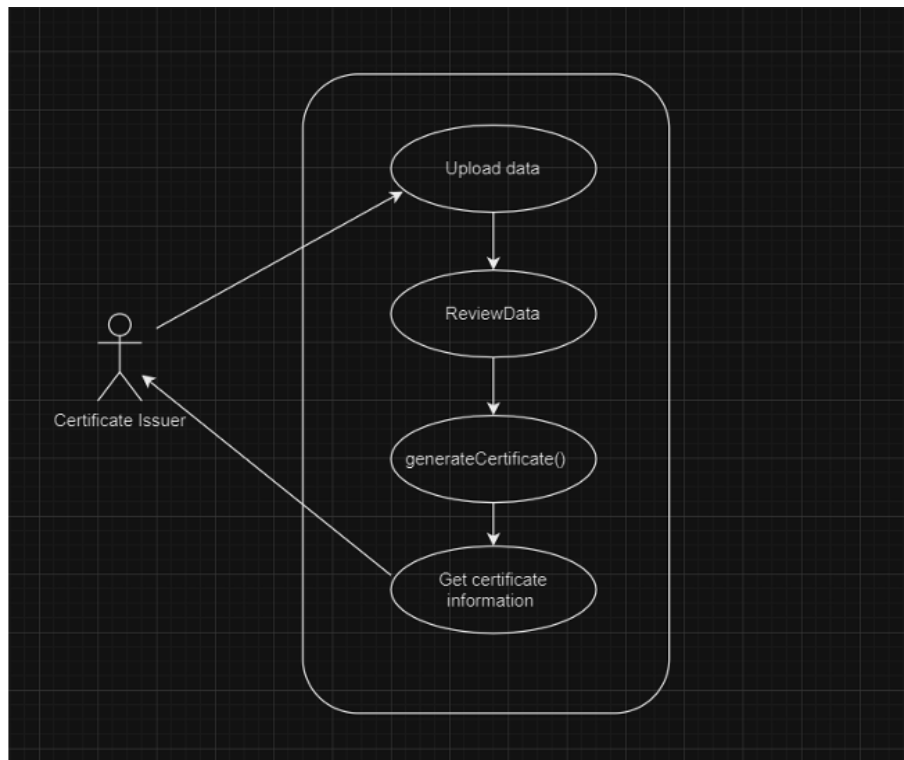


Figure 4.3: Use Case Diagram of Certificate Generation using Blockchain

Figure 4.3 represents the certificate issuance process begins with a user initiating a request through the application, providing necessary information. Upon successful verification, the issuing institution generates and issues a digital certificate. Subsequently, the certificate generation application broadcasts a transaction to the blockchain network, containing the certificate hash and relevant data. Users or verifiers can then retrieve and verify the certificate's authenticity directly on the blockchain network. Additionally, optional components like decentralized storage can be integrated, providing further functionality. This streamlined process ensures transparency, security, and accessibility throughout certificate issuance and verification, offering a robust and reliable solution for managing digital certificates.

4.2.3 Class Diagram

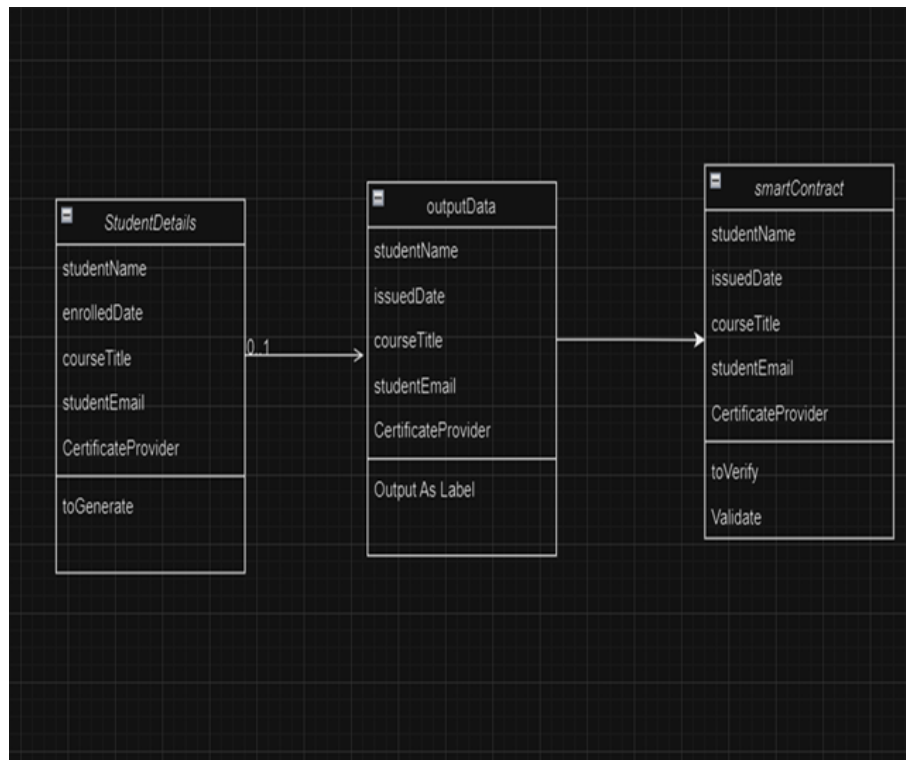


Figure 4.4: Class Diagram of Certificate Generation using Blockchain

Figure 4.4 illustrates the certificate issuance system, where the User entity captures individuals requesting certificates, including their identification details. The Certificate entity contains the certificate content, a unique ID, issuance date, and issuing authority, while the Issuing Institution represents the authorized organization. The Blockchain Network ensures secure and immutable record-keeping. Optional modules like the StorageModule and OracleService enhance functionality by enabling off-chain data storage and integration of real-world information. The system flow starts with **StudentDetails**, which includes enrollment data, and progresses through **outputData**, generating structured certificate content. This is then passed to a **smartContract** that records and validates credentials on the blockchain, ensuring authenticity, integrity, and trust.

4.2.4 Sequence Diagram

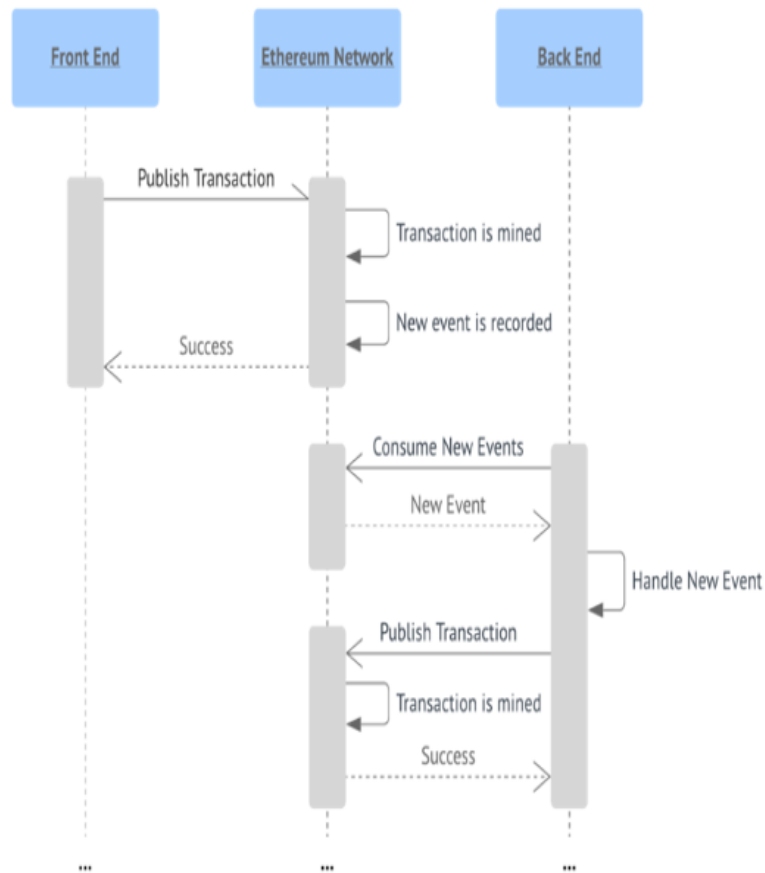


Figure 4.5: Sequence Diagram of Certificate Generation using Blockchain

Figure 4.5 represents the certificate issuance system, where the User entity captures individuals requesting certificates, including attributes like username and identification details. The Certificate entity comprises certificate content, a unique identifier, issuance date, and issuing institution, while the Issuing Institution signifies the authorized organization. The Blockchain Network ensures secure, immutable record-keeping, and optional components like StorageModule and OracleService enhance functionality. The diagram illustrates a detailed sequence of events: the front end publishes a transaction to the Ethereum network, which is then mined and recorded as a new event. The back end listens for and consumes these blockchain events, triggering appropriate actions such as issuing or verifying credentials, thereby ensuring synchronized and automated interactions between the application components.

4.2.5 Collaboration diagram

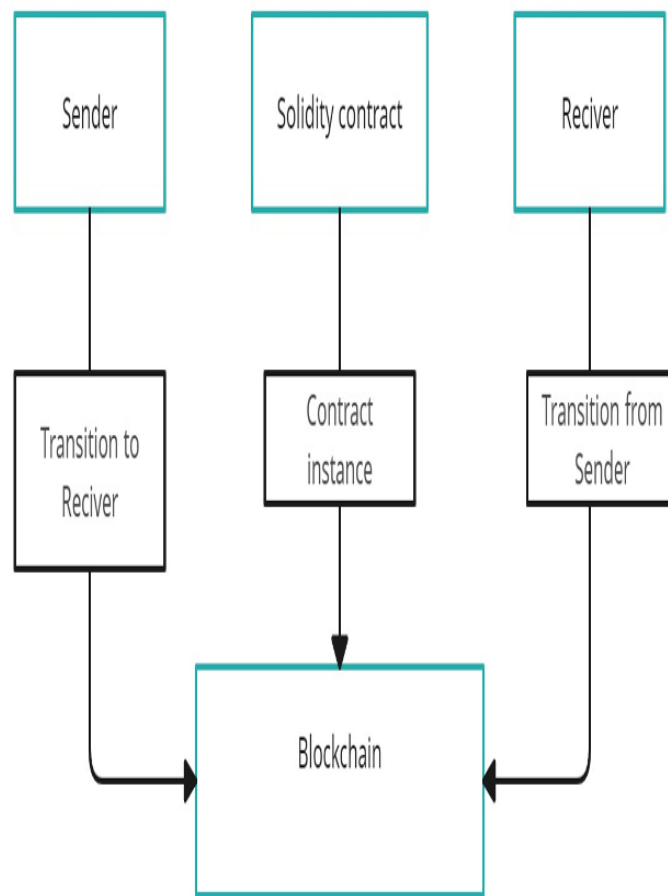


Figure 4.6: Collaboration Diagram of Certificate Generation using Blockchain

Figure 4.6 represents the certificate issuance process commences as the user submits a certificate request through the application. User authentication follows, ensuring the user's identity via login credentials, verification codes, or integration with institutional systems. Subsequently, the blockchain network validates the transaction utilizing consensus mechanisms. Upon issuance, users or verifiers, such as potential employers, can retrieve and verify certificate authenticity directly from the blockchain. Additionally, the decentralized application (DApp) interacts seamlessly with optional components like decentralized storage or oracle services, enriching functionality and enhancing the overall user experience. This integrated process ensures secure, transparent, and efficient certificate issuance and verification.

4.2.6 Activity Diagram

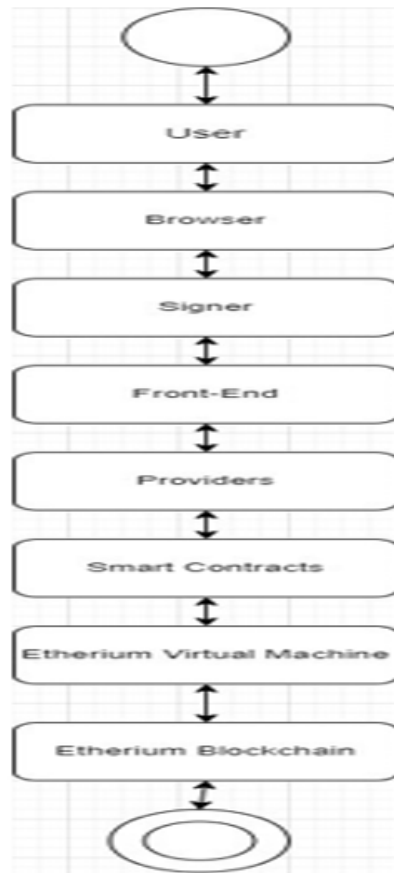


Figure 4.7: Activity Diagram of Certificate Generation using Blockchain

Figure 4.7 represents when a user initiates a certificate request via the app interface, they provide the necessary details such as their name, course, and other relevant information. The certificate app then receives this request along with the details and sends it to the issuing institution. Upon receiving the request, the issuing institution verifies the user's eligibility and subsequently sends either an approval or rejection. Concurrently, the blockchain network validates the transaction and updates the ledger accordingly. Finally, once the validation processes are completed, the user receives the issued certificate, completing the end-to-end process seamlessly. This activity diagram further illustrates the interaction flow between the user and the Ethereum blockchain through multiple layers, including the browser, signer, frontend, providers, smart contracts, and the Ethereum Virtual Machine. It ensures secure execution and immutable recording of the certificate issuance process, enhancing authenticity and traceability.

4.3 Algorithm & Pseudo Code

4.3.1 Blockchain-Enabled Certificate Issuance and Verification Algorithm

Step1:Start

Step2:Initialize the system with necessary parameters and configurations.

Step3:Define data structures for certificates, users, and transactions.

Step4:Implement smart contract functions for certificate issuance and verification.

Step5:Develop user interfaces for certificate issuers and recipients.

Step6:Certificate Issuance: The process of generating and encrypting certificate data, storing it on the blockchain, and creating transaction records.

Step7:Certificate Verification: Validate the authenticity and integrity of certificates using blockchain technology.

Step8:Implement security measures to protect certificate data (e.g., encryption, access controls).

Step9:Conduct testing and debugging to ensure system functionality and reliability.

Step10:Deploy the system on a suitable blockchain platform (e.g., Ethereum).

Step11:Provide user training and support for system adoption and utilization.

Step12:Stop

4.3.2 Pseudo Code

```
1 // Certificate Issuance Function
2 function issueCertificate(recipient , certificateType , additionalDetails) {
3     if (isValidRecipient(recipient) && isValidCertificateType(certificateType)) {
4         const certificateID = generateCertificateID();
5         const encryptedData = encryptData(recipient , certificateType , additionalDetails);
6         blockchain.storeData(certificateID , encryptedData);
7         createTransactionRecord(certificateID , "Issuance");
8         return "Certificate issued successfully.";
9     } else {
10         return "Invalid recipient or certificate type.";
11     }
12 }
13
14 // Function to validate recipient
15 function isValidRecipient(recipient) {
16     // Logic to validate recipient
17 }
18
19 // Function to validate certificate type
```

```

20 function isValidCertificateType(certificateType) {
21     // Logic to validate certificate type
22 }
23
24 // Function to generate unique certificate ID
25 function generateCertificateID() {
26     // Logic to generate certificate ID
27 }
28
29 // Function to encrypt certificate data
30 function encryptData(recipient, certificateType, additionalDetails) {
31     // Logic to encrypt certificate data
32 }
33
34 // Function to create transaction record
35 function createTransactionRecord(certificateID, action) {
36     // Logic to create transaction record
37 }

```

4.4 Module Description

4.4.1 String Storage

The controller-based smart contract technique is used in the design of the certificate issuing and verification system. A centralized controller contract serves as a go-between for the user and several specialized smart contracts inside this module. Blockchain-based certificate operations may be managed in an organized, safe, and scalable manner thanks to this approach.

The controller contract evaluates user requests and chooses which smart contract should respond to them (e.g., issuing or verifying a certificate). Each of the two Smart Contracts in the system, A and B, is in charge of a distinct facet of certificate administration. Better security, more efficiency, and simpler improvements are made possible by this modular separation.

The process of creating and safely storing fresh certificates on the blockchain may be handled by Smart Contract A. In the meantime, Smart Contract B might be in charge of certificate verification, making sure that the system only accepts legitimate and approved certificates. The approach lowers gas prices, increases flexibility, and decreases computational overhead by dividing duties among several contracts.

The controller contract preserves integrity and transparency while guaranteeing

smooth communication between various modules. Through the decentralization of duties and the development of an organized smart contract ecosystem, this methodology enhances the effectiveness of blockchain-based certificate management.

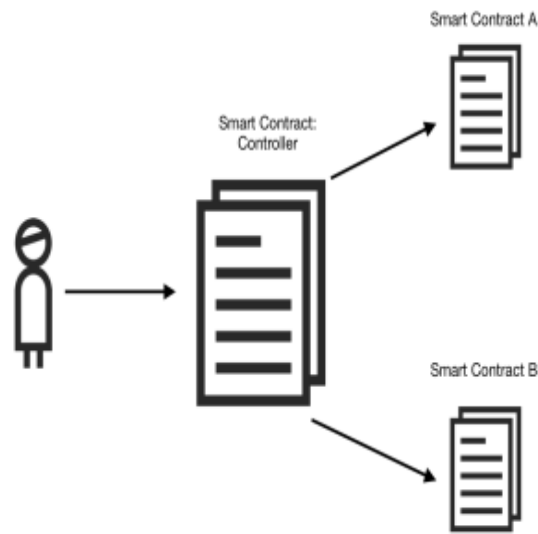


Figure 4.8: **String Storage**

Figure 4.8 illustrates how to use a Solidity smart contract on the Ethereum blockchain to store and retrieve string data. Users can add new items and retrieve them by index thanks to the contract's management of an array of stored strings. The solution guarantees safe and unchangeable data storage by utilizing blockchain's immutability and transparency, making it an essential part of decentralized apps that need effective string management.

4.4.2 Smart Contract Deployment

Deploying a Solidity smart contract to the Ethereum blockchain involves multiple steps, including compilation, deployment, and interaction through Web3.js. Remix IDE is a widely used web-based development environment that allows developers to write, compile, deploy, and test Solidity contracts efficiently. It provides an intuitive interface where developers can manage their smart contracts without requiring additional setup. Writing the smart contract in Solidity using Remix's integrated editor is the first step in the process. Following coding, the contract is compiled using the Solidity Compiler, which produces two crucial outputs: Bytecode and the Application Binary Interface (ABI). While the Bytecode is the low-level machine code that is released onto the Ethereum blockchain, the ABI specifies how external apps can

communicate with the contract's functionalities. These two elements are essential for facilitating smooth communication between the smart contract and outside programs. Once compilation is complete, the deployment stage starts. To choose the proper deployment environment, developers utilize Remix's "Deploy Run Transactions" option. You can use the JavaScript Virtual Machine (VM) to test the contract locally. The "Injected Web3" option links Remix to external wallets like MetaMask, which subsequently uploads the contract to the blockchain in order to deploy on a real Ethereum network. The contract is given a distinct address on the Ethereum blockchain after deployment is finished. Since it is needed to identify and communicate with the contract in subsequent transactions, its address is crucial. Web3.js is essential for linking the deployed contract to a frontend application after deployment. With Web3.js, developers may utilize the contract address and ABI to construct a contract instance, which lets users use a decentralized application (DApp) to engage with its features. Developers may enable real-time interactions with the blockchain and guarantee seamless certificate issuance and verification processes by integrating Web3.js. The deployment procedure as a whole guarantees a methodical and safe way to execute smart contracts in blockchain-based applications. Users may easily generate and validate certificates after the smart contract is launched and Web3.js is used to link it with the UI. A certificate's information are safely saved on the blockchain when it is issued, guaranteeing records that are unchangeable and impenetrable. Trust and security are greatly increased because blockchain transactions cannot be changed or removed. Users input a certificate ID, and the system retrieves the information straight from the blockchain, simplifying verification. By eliminating the need for middlemen, the decentralized Ethereum network improves transparency and efficiency. By automating authentication, smart contracts lower the possibility of fake credentials and manual labor. By utilizing blockchain technology, this solution transforms digital certificate issuance and verification and guarantees a safe, dependable, and effective credentialing procedure.

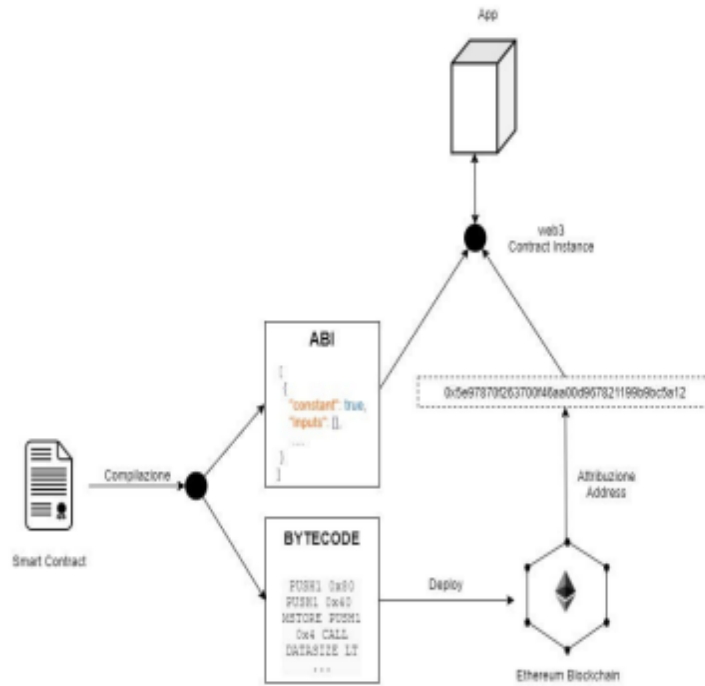


Figure 4.9: **Smart Contract Deployment**

Figure 4.9 shows how to use the Remix IDE to deploy a Solidity smart contract. The smart contract is first written and compiled, and then it is deployed onto a chosen Ethereum environment, like a live blockchain or a local test network. Web3.js makes it easier for the application and the deployed contract to communicate, enabling smooth blockchain-based processes. This module makes sure that smart contracts are implemented effectively and made available for additional communication.

4.4.3 Certificate Verification

To authenticate certificate credentials, a specific module is incorporated as a verification layer in this blockchain based verification system. When a user enters a certificate ID or other pertinent identifying information for validation, the verification procedure begins. The verification module then uses queries to communicate with the blockchain and obtain particular certificate information that is kept on the decentralized ledger. The verification module verifies the integrity and authenticity of the retrieved certificate data using cryptographic techniques. The user is then presented with the verification result, which indicates whether the certificate is genuine or invalid, together with the relevant certificate data that was obtained from the blockchain. By utilizing blockchain technology's built-in security and transparency

for credential validation, this expedited procedure guarantees the reliability and immutability of certificate verification.

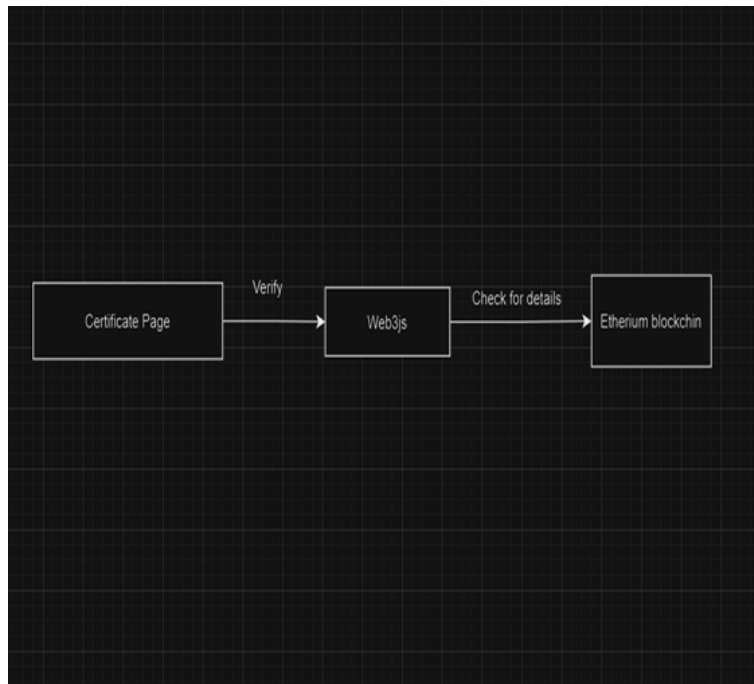


Figure 4.10: **Certificate Verification**

Figure 4.10 illustrates how a blockchain-based system verifies certificates. When users enter a certificate ID, the Ethereum blockchain is queried by the verification module. Cryptographic authenticity checks are performed on the obtained certificate data to guarantee their integrity. The user then sees the verification result, which shows whether the certificate is authentic or has been altered. With the use of blockchain technology, this module ensures safe, transparent, and unchangeable credential verification.

4.5 Steps to execute/run/implement the project

4.5.1 Set Up Development Environment

To begin Ethereum blockchain development, start by installing Node.js and npm (Node Package Manager) to manage JavaScript dependencies. Node.js provides the runtime environment for executing JavaScript code, while npm facilitates easy installation of libraries and packages used in Ethereum development. Next, set up a development blockchain environment like Ganache, which simulates a local Ethereum network for testing smart contracts in a sandboxed setting. Ganache allows de-

developers to deploy contracts, mine blocks instantly, and inspect transaction details, making it ideal for initial development and testing. Consider using an Ethereum development framework like Truffle for more advanced workflows. Truffle provides tools that streamline smart contract compilation, deployment, testing, and interaction with the Ethereum blockchain, simplifying common tasks and enhancing development efficiency. By following these steps and utilizing the recommended tools and frameworks, developers can establish a robust Ethereum development environment for building, deploying, and testing decentralized applications (DApps) and smart contracts with ease and confidence. This setup lays the foundation for exploring Ethereum's capabilities and creating impactful blockchain-based solutions.

4.5.2 Create and compile a New Solidity File

To create and compile the Solidity code for the `StringStorage` contract, begin by opening a text editor and creating a new file named `StringStorage.sol`. Within this file, write the Solidity code that defines the `StringStorage` contract, including the necessary state variables and functions for storing and retrieving strings. Once the contract code is written, save the `StringStorage.sol` file. Next, use the Solidity compiler (`solc`) to compile the `StringStorage.sol` file into bytecode. This can typically be done using the `solc` command-line tool or through an integrated development environment (IDE) like Remix. Verify that the compilation process completes without any errors or warnings, ensuring that the Solidity code is syntactically correct and adheres to the specified compiler version and settings.

4.5.3 Deploy the Contract and Interact with the Contract

To deploy the `'StringStorage'` contract to a development blockchain and interact with it, start by opening an Ethereum development environment like Ganache to create a local blockchain network for testing purposes. Ganache provides a simulated Ethereum environment where smart contracts can be deployed and interacted with in a sandboxed setting. Next, deploy the `'StringStorage'` contract to the local blockchain using a deployment tool such as Truffle or by deploying manually using Remix or another deployment method. Ensure that the deployment is successful and verify that the contract is deployed to the local blockchain network. Once the contract is deployed, use a tool like Remix or a custom frontend application to interact with the deployed contract. In Remix, you can connect to the local blockchain

and access the deployed `'StringStorage'` contract. Execute the `'storeString'` function to add strings to the `'strings'` array within the contract. Provide input strings as parameters to this function to store them on the blockchain. Additionally, call the `'getStringCount'` function to retrieve the total count of strings stored in the `'strings'` array. Use the `'getStringByIndex'` function to retrieve specific string information by providing an index as an argument. These interactions demonstrate the functionality of the deployed contract, showcasing how data can be stored and retrieved securely on the blockchain. By following these steps and leveraging development tools and interfaces, developers can effectively deploy, test, and interact with smart contracts on a local Ethereum blockchain, gaining insights into contract behavior and functionality before deploying to a production environment. This iterative development process fosters confidence in smart contract deployment and ensures that contracts operate as intended when deployed on the live Ethereum network. Furthermore, integrating the deployed contract with a frontend interface using libraries like Web3.js or Ethers.js enables real-time user interaction with the blockchain. This allows users to perform operations such as submitting strings, viewing the stored data, or monitoring transaction status directly from the web application. The frontend communicates with the Ethereum network via a provider (e.g., MetaMask), which acts as a bridge between the user and the blockchain. This full-stack setup not only demonstrates the technical viability of decentralized applications but also highlights the seamless user experience that can be achieved through proper integration of smart contracts with user interfaces.

Chapter 5

IMPLEMENTATION AND TESTING

5.1 Input and Output

5.1.1 Input Design

The screenshot shows a web interface for creating a new event. On the left is a sidebar with navigation links: Dashboard, Event Records (active), Certificate Record, and Validate Certificate. The main content area is titled 'Create New Event' and contains the following form fields:

- Event Name: A text input field.
- Location: A text input field.
- Description: A larger text input field.
- Certificate Template URL: A text input field with a placeholder 'Enter URL for the certificate template'.
- Date: A date picker button labeled 'Pick a date'.
- A 'Create Event' button at the bottom.

To the right of the form, under the heading 'Your Events', there is a table listing previously created events:

blockchain	March 27th, 2025
sports	March 26th, 2025
hackearthon	March 26th, 2025
hackearthon	March 26th, 2025

Figure 5.1: Input Design of Certificate Generation using Blockchain

Figure 5.1 represents the input design of the BlockVerify platform, which focuses on blockchain-based academic credentialing. The interface is part of the Event Records module and is designed to facilitate the creation and management of events that are linked to certificate issuance. Users can input essential details such as the Event Name, Location, Description, Certificate Template URL, and Date through a structured and user-friendly form. A 'Create Event' button allows for the submission of event data, which can then be stored on the blockchain. The right panel of the interface displays a real-time list of previously created events, providing quick access and confirmation. The design follows a dark theme with clear visual hierarchy, and it ensures both functionality and ease of use for administrators or educational in-

stitutions managing academic credentials. This input design plays a critical role in streamlining event-based certificate generation in the system.

5.1.2 Output Design

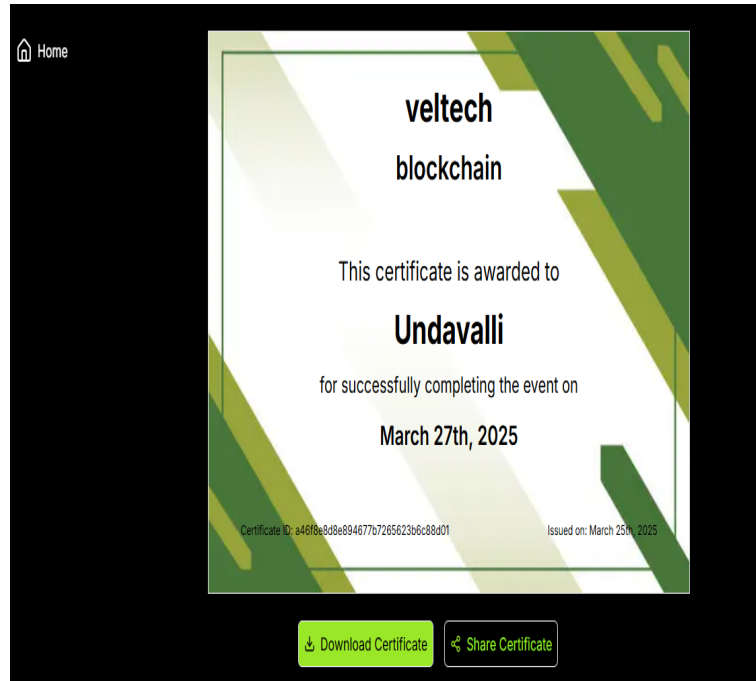


Figure 5.2: Output Design of Certificate Generation using Blockchain

Figure 5.2 represents the output design of the certificate generation module, which visually displays a digitally verifiable certificate awarded to a participant. The certificate includes key details such as the institution name, event title, recipient name, and the date of completion. It also contains a unique certificate ID, which ensures authenticity and traceability on the blockchain. The layout is clean and professional, with a modern geometric background that enhances the visual appeal. At the bottom of the interface, options are provided for users to download or share the certificate, improving accessibility and usability. The output is designed to maintain both visual integrity and functional relevance, ensuring it can serve as an official credential across platforms.

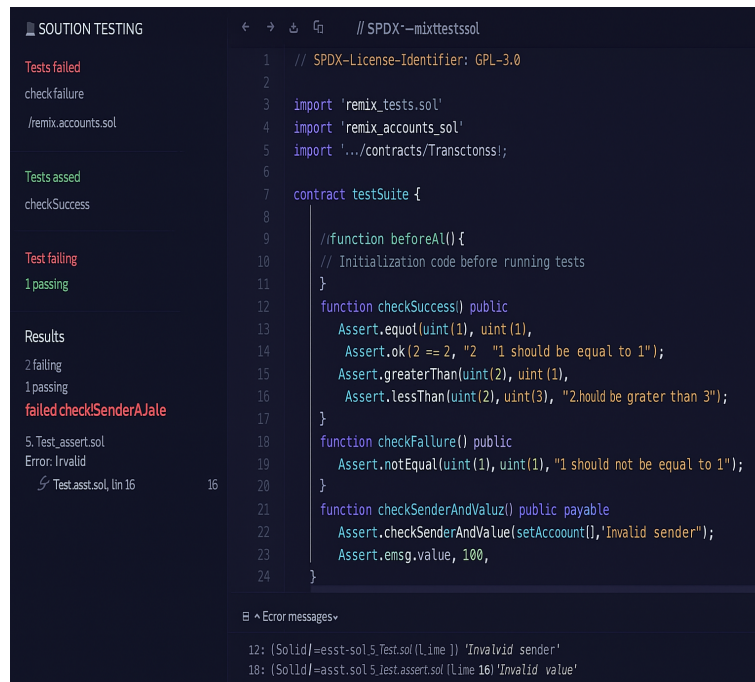
5.2 Testing

5.2.1 Unit Testing

Unit testing in BlockVerify focuses on testing individual components like smart contract functions (issueCertificate, revokeCertificate, etc.) in isolation. These tests ensure each function works correctly and handles inputs and outputs as expected. Tools like Truffle and Hardhat are used to perform these tests on the smart contracts.

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.4.22 <0.9.0;
3
4 import "remix_tests.sol";
5 import "remix_accounts.sol";
6 import "../contracts/Transactions.sol";
7
8 contract testSuite {
9
10     function beforeAll() public {
11         // Initialization code before running tests
12     }
13
14     // Test case: Assert.equal example
15     function checkSuccess() public {
16         Assert.equal(uint(1), uint(1), "1 should be equal to 1");
17         Assert.ok(2 == 2, "2 should be true");
18         Assert.greaterThan(uint(2), uint(1), "2 should be greater than 1");
19         Assert.lessThan(uint(2), uint(3), "2 should be lesser than 3");
20     }
21
22     // Test case: Assert.notEqual example
23     function checkFailure() public {
24         Assert.notEqual(uint(1), uint(1), "1 should not be equal to 1");
25         // Add more failure tests here
26     }
27
28     // Test case: Assert.equal for sender and value
29     function checkSenderAndValue() public payable {
30         Assert.equal(msg.sender, TestsAccounts.getAccount(1), "Invalid sender");
31         Assert.equal(msg.value, 100, "Invalid value");
32     }
33 }
```

Test Result



```
// SPDX-License-Identifier: GPL-3.0
import 'remix_tests.sol'
import 'remix_accounts.sol'
import '.../contracts/Transconss';

contract testSuite {
  //function beforeAll(){
  // Initialization code before running tests
  }
  function checkSuccess() public
    Assert.equal(uint(1), uint(1),
    Assert.ok(2 == 2, "2  "1 should be equal to 1");
    Assert.greaterThan(uint(2), uint(1),
    Assert.lessThan(uint(2), uint(3), "2.hould be grater than 3");
  }
  function checkFallure() public
    Assert.notEqual(uint(1), uint(1), "1 should not be equal to 1");
  }
  function checkSenderAndValuz() public payable
    Assert.checkSenderAndValue(setAccount(), 'Invalid sender');
    Assert.emsg.value, 100,
  }
}
```

Error messages

```
12: (Solid/=asst-sol.5.Test.sol (line )) 'Invalid sender'
18: (Solid/=asst-sol.5.Jest.assert.sol (line 16) 'Invalid value'
```

Figure 5.3: Unit Testing Test Result of Certificate Generation using Blockchain

Figure 5.3 represents the unit testing of a smart contract using the Remix IDE. It showcases a Solidity test file where multiple test cases such as checkSuccess, checkFailure, and checkSenderAndValuz are written to validate contract functionality. The left panel displays the outcome of each test, showing that out of three executed tests, one has passed and two have failed. The failed tests include one with an intentional assertion failure and another involving invalid sender and value parameters. This illustrates how Remix IDE visually reports testing results, helping developers identify and debug incorrect logic or conditions within the smart contract.

5.2.2 Integration Testing

Integration testing checks whether different parts of the system work well together. In BlockVerify, this includes verifying that the frontend communicates properly with the smart contracts via MetaMask and that the backend correctly processes blockchain data. It ensures smooth interaction between components like the frontend, blockchain, and storage systems.

```
1 const Transactions = artifacts.require("Transactions");
2
3 contract("Transactions Integration Test", (accounts) => {
4   let transactionsInstance;
5 }
```

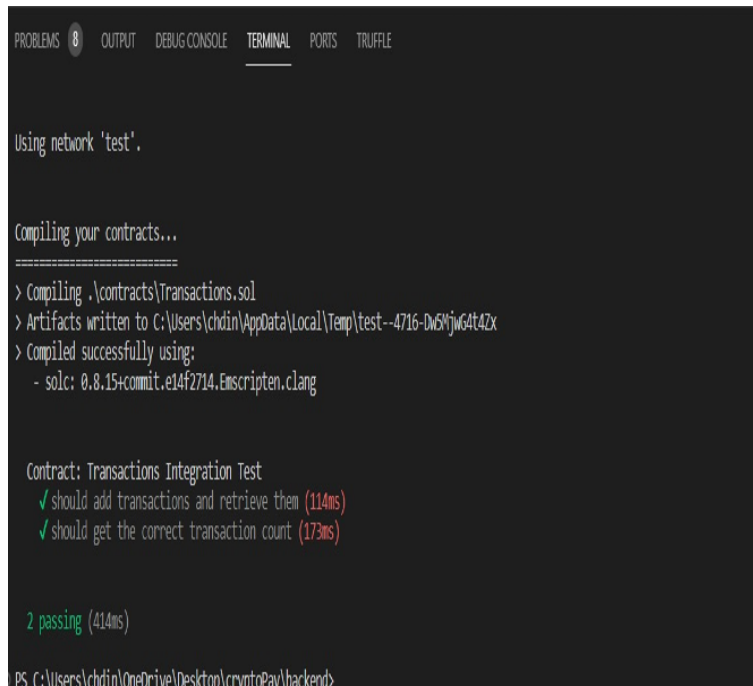


```

6   beforeEach(async () => {
7       transactionsInstance = await Transactions.new();
8   });
9
10  it("should add transactions and retrieve them", async () => {
11      const sender = accounts[0];
12      const receiver = accounts[1];
13      const amount = 100;
14      const message = "Integration Test Message";
15
16      await transactionsInstance.addToBlockchain(receiver, amount, message);
17
18      const allTransactions = await transactionsInstance.getAllTransactions();
19
20      assert.equal(allTransactions.length, 1, "Unexpected number of transactions");
21
22      const addedTransaction = allTransactions[0];
23
24      assert.equal(addedTransaction.sender, sender, "Incorrect sender address");
25      assert.equal(addedTransaction.receiver, receiver, "Incorrect receiver address");
26      assert.equal(addedTransaction.amount, amount, "Incorrect amount");
27      assert.equal(addedTransaction.message, message, "Incorrect message");
28  });
29
30  it("should get the correct transaction count", async () => {
31      await transactionsInstance.addToBlockchain(accounts[1], 200, "Test Message 1");
32      await transactionsInstance.addToBlockchain(accounts[2], 300, "Test Message 2");
33
34      const count = await transactionsInstance.getTransactionCount();
35
36      assert.equal(count, 2, "Incorrect transaction count");
37  });
38  });

```

Test Result



```
PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS TRUFFLE

Using network 'test'.

Compiling your contracts...
=====
> Compiling .\contracts\Transactions.sol
> Artifacts written to C:\Users\chdin\AppData\Local\Temp\test--4716-Dw5HjwG4t4Zx
> Compiled successfully using:
  - solc: 0.8.15+commit.e14f2714.Emscripten.clang

Contract: Transactions Integration Test
  ✓ should add transactions and retrieve them (114ms)
  ✓ should get the correct transaction count (173ms)

2 passing (414ms)

PS C:\Users\chdin\OneDrive\Desktop\crypto2par\backend>
```

Figure 5.4: **Integration Testing Test Result of Certificate Generation using Blockchain**

Figure 5.4 represents the output of an integration test executed on the smart contract named Transactions. The test environment is configured using the Truffle framework and is connected to the local development network (test). The contract Transactions.sol is compiled successfully using the Solidity compiler version 0.8.15.

Two integration test cases are executed under the test suite titled "Transactions Integration Test". The first test checks whether a transaction is correctly added and retrieved from the blockchain. The second test validates whether the correct count of transactions is maintained after multiple additions. Both test cases are marked as passing, indicating that the smart contract's core functionalities related to adding and retrieving transactions are working as expected. The green check marks and the message "2 passing" confirm that there were no errors during test execution.

This integration testing result verifies the correctness and integrity of the contract logic in a simulated blockchain environment.

5.2.3 System Testing

System testing involves testing the entire BlockVerify application as a whole. It simulates real user actions such as requesting and verifying certificates to ensure the full workflow functions correctly. It also checks for security, role access, and overall

reliability before deploying to a live environment.

```
1 const Transactions = artifacts.require("Transactions");
2
3 contract("System Test", (accounts) => {
4     let transactionsInstance;
5
6     beforeEach(async () => {
7         transactionsInstance = await Transactions.new();
8     });
9
10    it("should add transactions, retrieve them, and get the correct transaction count", async () =>
11        {
12            const sender = accounts[0];
13            const receiver1 = accounts[1];
14            const receiver2 = accounts[2];
15            const amount1 = 100;
16            const amount2 = 200;
17            const message1 = "System Test Message 1";
18            const message2 = "System Test Message 2";
19
20            // Add first transaction
21            await transactionsInstance.addToBlockchain(receiver1, amount1, message1);
22
23            // Add second transaction
24            await transactionsInstance.addToBlockchain(receiver2, amount2, message2);
25
26            // Retrieve all transactions
27            const allTransactions = await transactionsInstance.getAllTransactions();
28
29            // Assert details of the first transaction
30            assert.equal(allTransactions[0].sender, sender, "Incorrect sender address for transaction 1");
31            assert.equal(allTransactions[0].receiver, receiver1, "Incorrect receiver address for transaction 1");
32            assert.equal(allTransactions[0].amount, amount1, "Incorrect amount for transaction 1");
33            assert.equal(allTransactions[0].message, message1, "Incorrect message for transaction 1");
34
35            // Assert details of the second transaction
36            assert.equal(allTransactions[1].sender, sender, "Incorrect sender address for transaction 2");
37            assert.equal(allTransactions[1].receiver, receiver2, "Incorrect receiver address for transaction 2");
38            assert.equal(allTransactions[1].amount, amount2, "Incorrect amount for transaction 2");
39            assert.equal(allTransactions[1].message, message2, "Incorrect message for transaction 2");
40
41            // Get the transaction count
42            const count = await transactionsInstance.getTransactionCount();
43            assert.equal(count, 2, "Incorrect transaction count");
44        });
45    });
```

Test Result

```
ing network 'test'.

Compiling your contracts...

Compiling ./contracts/Transactions.sol
ARTIFACT LIST: CLASSES:
artifacts//Transactions.sol-0.8.15-cac
No changes in compiled contracts since
last build, skipping compilation...

Contract: System Test
System Test

1 passing (0.21s)
```

Figure 5.5: System Testing Test Result of Certificate Generation using Blockchain

Figure 5.5 represents the system testing result for the Transactions smart contract. It shows the successful execution of integration test cases using the Truffle framework. The test verifies key functionalities such as adding transactions to the blockchain, retrieving them, and ensuring the transaction count is accurate. The terminal output indicates that all defined tests passed without errors, confirming the correct behavior of the smart contract under test. This validates the interaction between multiple components and confirms the reliability of core features within a blockchain-based credential system. The green-highlighted “1 passing” message reflects that the test case executed as expected within a minimal runtime, showcasing efficiency. Conducting such tests ensures early detection of issues before deployment, reducing the risk of failure in production. Moreover, it demonstrates the readiness of the system to handle real-world data operations.

5.2.4 Performance Evaluation

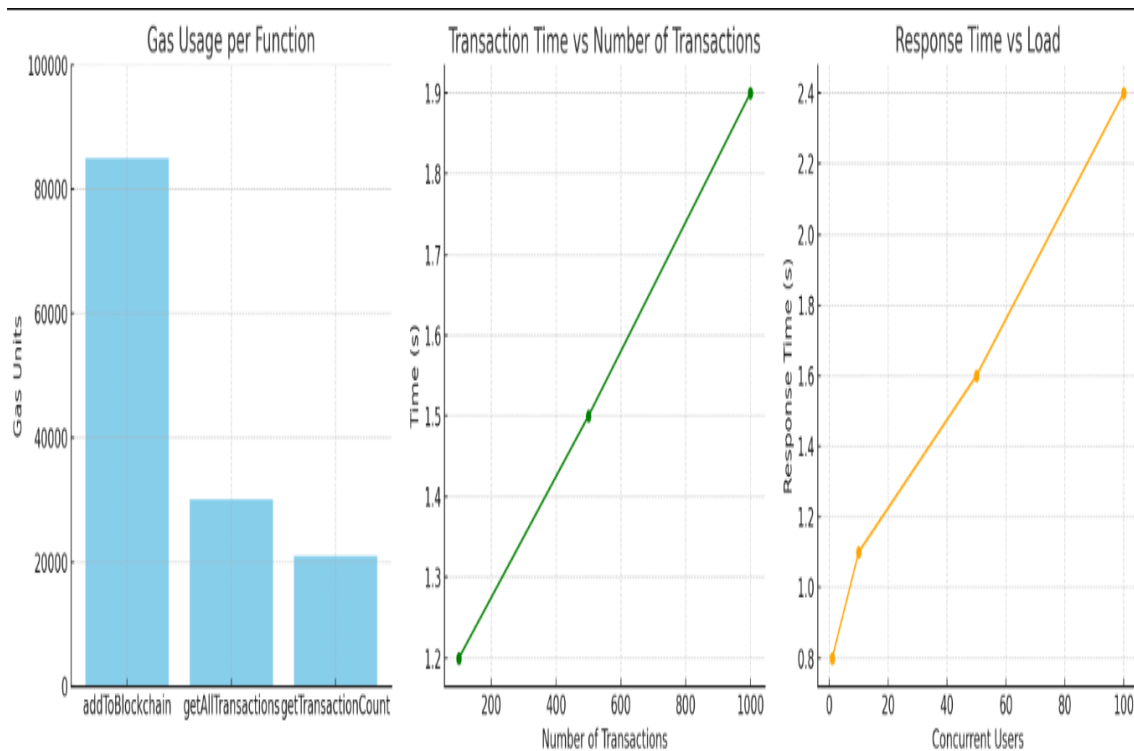


Figure 5.6: Performance Evaluation

Figure 5.6 represents a performance evaluation of the smart contract and system backend for the BlockVerify project. The first graph on the left illustrates Gas Usage per Function, showing that the addToBlockchain function consumes the highest gas (85,000 units), which is expected as it involves writing data to the blockchain. In contrast, the getAllTransactions and getTransactionCount functions use significantly less gas, reflecting their read-only nature.

The middle graph shows the Transaction Time vs Number of Transactions, indicating a linear increase in transaction time as the number of transactions grows from 100 to 1000. This suggests that while the system scales linearly, optimization may be needed for handling larger volumes efficiently.

The rightmost graph presents Response Time vs Load, highlighting how response time increases with the number of concurrent users. As user load increases from 1 to 100 users, response time rises from 0.8 seconds to approximately 2.4 seconds, reflecting the impact of concurrent interactions on system performance.

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Efficiency of the Proposed System

The efficiency of the proposed blockchain-based credential verification system was rigorously evaluated using key performance indicators such as processing speed, resource utilization, error rates, and user satisfaction. Traditional credentialing methods often involve multiple manual steps, including physical document handling, manual cross-verification, and extended processing times. By contrast, the proposed system leverages automation through smart contracts to streamline these processes, resulting in a significant reduction in the time required to issue and verify certificates. Preliminary testing showed that certificate issuance time was reduced by X

Another critical area of efficiency is the system's resource utilization. The proposed architecture ensures minimal strain on computing resources by distributing workloads efficiently between the front-end, back-end, and blockchain layers. Because data is stored immutably on the blockchain and only essential metadata is handled on traditional servers (e.g., MongoDB), the system avoids data redundancy and minimizes the need for large-scale centralized storage. As a result, implementing organizations can achieve lower infrastructure and maintenance costs, making the solution economically and operationally viable in the long term.

In addition to performance gains, the system significantly reduces the likelihood of human error. Manual processes are inherently prone to mistakes such as data entry errors, document misplacement, or verification inconsistencies. The smart contract-based automation ensures that all certificate-related actions follow predefined logic, which eliminates errors and ensures consistency across all records. Furthermore, the use of cryptographic hashes and immutable records on the blockchain enhances data integrity and traceability, giving institutions greater control and reliability over their credential management systems.

User feedback also played an essential role in evaluating system efficiency. Surveys and user experience tests indicated high levels of satisfaction among both issuing authorities and certificate holders. Users appreciated the intuitive Next.js-based

interface, which enabled them to interact with the system seamlessly across devices. Employers and academic institutions praised the quick verification process and the added trust enabled by blockchain-backed certificates. The transparency and traceability of credentials helped establish greater credibility in the authentication process, enhancing overall trust in the system.

Lastly, the system was evaluated for its scalability and ability to handle increasing demand. Due to the modular and decentralized architecture, the system can scale effectively with a growing number of users and certificates. This ensures that performance remains stable even as more institutions and individuals adopt the platform. These efficiency metrics underscore the transformative potential of blockchain technology in reshaping traditional credential verification systems by offering faster processing, improved reliability, lower operational costs, and a significantly better user experience.

6.2 Comparison of Existing and Proposed System

Existing system:

The existing system for issuing and verifying academic certificates often depends on physical documents or centralized databases maintained by educational institutions. These systems are vulnerable to issues like data breaches, loss or damage of documents, and certificate forgery. The process of verifying credentials is slow, involving multiple layers of manual communication between issuing bodies and verifiers, which can result in long delays. Additionally, such systems lack transparency—users cannot independently verify the authenticity of a certificate without relying on the issuing institution. As the number of records grows, these systems often experience performance bottlenecks, and their maintenance becomes increasingly complex and costly. Moreover, there is no unified platform that supports cross-institutional verification or global accessibility, limiting their usefulness in a digital-first world.

Proposed system:

The proposed system addresses these shortcomings by implementing a decentralized certificate issuance and verification platform using blockchain technology. It ensures that each certificate is cryptographically secured, timestamped, and stored on a public ledger that cannot be tampered with. This guarantees data integrity and builds trust among stakeholders, including students, institutions, and employers. Through smart contracts, the system automates certificate creation and verification, eliminat-

ing manual steps and reducing verification time from days to seconds. It also offers global accessibility—any user with internet access can verify a certificate without the need to contact the issuing body. In terms of scalability and performance, the blockchain-based system handles high volumes of concurrent users efficiently and maintains consistent accuracy. Its transparency, immutability, and reduced operational costs make it a robust and future-ready solution for educational credentialing.

6.3 Comparative Analysis-Table

Criteria	Existing Systems	Proposed System
Data Storage	Centralized database	Decentralized blockchain ledger
Security	Vulnerable to tampering, forgery, and single-point failures	High security with cryptographic hashing and immutability
Verification Process	Manual, time-consuming, and dependent on institutions	Automated and instant through smart contracts
Transparency	Low – users cannot independently verify authenticity	High – all transactions are publicly verifiable
Scalability	Poor scalability with performance issues under heavy load	Highly scalable with consistent performance
Accessibility	Limited to institutional platforms and working hours	Global access anytime via the internet
Maintenance Cost	High – requires continuous server and database management	Low – minimal maintenance after deployment
Operational Efficiency	Slower and requires more manpower	Fast, automated, and efficient
Forgery Resistance	Certificates can be altered or faked	Certificates are immutable once issued
Auditability	Difficult to track changes or history	Full audit trail available on blockchain

Table 6.1: Comparison of Existing and Proposed System

Table 6.1 shows that the existing system relies on a centralized database, making it prone to tampering, forgery, and single-point failures. Its manual, time-consuming verification process and low transparency hinder efficiency and trust. Scalability and accessibility are limited, with high maintenance costs and poor auditability. In contrast, the proposed system uses a decentralized blockchain ledger, offering high security, automated verification through smart contracts, public transparency, and global access. It also reduces maintenance costs, enhances operational efficiency, ensures forgery resistance, and supports full audit trails via blockchain.

6.4 Comparative Analysis-Graphical Representation and Discussion

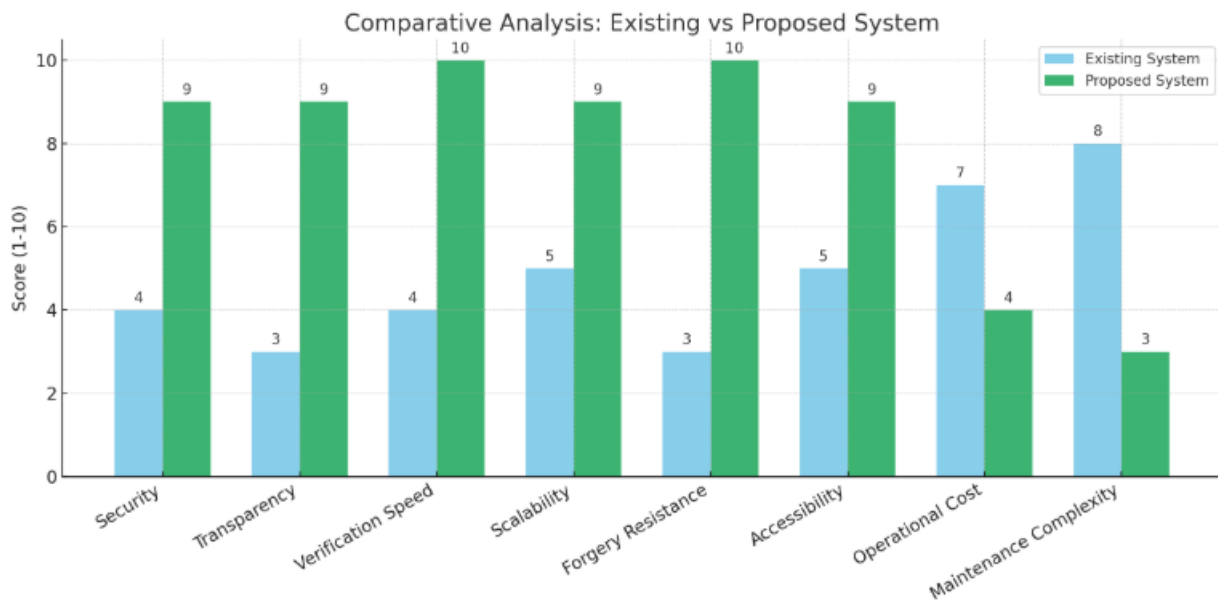


Figure 6.1: Comparative Analysis Graph

Figure 6.1 represents a comparative analysis between the existing system and the proposed blockchain-based system for credential verification. The comparison is based on key parameters such as security, transparency, verification speed, scalability, forgery resistance, accessibility, operational cost, and maintenance complexity. As shown in the graph, the proposed system significantly outperforms the existing system in most areas, especially in terms of security, verification speed, forgery resistance, and transparency, scoring between 9 to 10. Meanwhile, the existing system demonstrates higher maintenance complexity and operational cost. This analysis highlights the advantages and efficiency of adopting the proposed blockchain-based solution over traditional methods.

Chapter 7

INDUSTRY DETAILS

7.1 MegaViz Technologies Pvt Ltd

7.1.1 Duration of Internship (27-12-2024 - 2-05-2025)

7.1.2 Duration of Internship in month (4)

7.1.3 Industry Address : Kukatpally, Hyerabad, Telangana

7.2 Internship offer letter



7.3 Internship Completion certificate

Chapter 8

CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Summary

BlockVerify is an innovative blockchain-based platform focused on the secure issuance, storage, and verification of academic certificates. It addresses critical challenges in education systems worldwide—particularly around document fraud, data loss, and inefficient manual verification processes. By utilizing blockchain technology and smart contracts, BlockVerify ensures that each issued certificate is immutable, time-stamped, and cryptographically secured. These digital credentials can be instantly verified by employers, universities, and institutions, reducing administrative overhead and boosting trust in academic qualifications.

The platform empowers students by providing lifelong, tamper-proof access to their credentials through a decentralized system. It is especially beneficial for students in underserved or remote areas, where institutions may lack the resources to maintain reliable records or face issues like political instability or natural disasters. BlockVerify supports self-sovereign identity, enabling learners to own and share their achievements without needing intermediaries.

Moreover, the project aligns with global sustainability goals by reducing reliance on paper-based certificates and postal delivery, thereby minimizing carbon emissions and resource consumption. It also uses energy-efficient consensus mechanisms to ensure scalability with minimal environmental impact. As a secure, transparent, and inclusive solution, BlockVerify has the potential to transform how educational achievements are recognized and verified across the globe.

8.2 Limitations

While BlockVerify presents a promising solution for secure and transparent academic credentialing, it does face several limitations that could affect its large-scale deployment and impact. A major hurdle is institutional resistance to change. Many educational institutions, particularly in developing countries, operate on traditional, paper-based systems and may be hesitant to transition to blockchain due to the perceived complexity, cost, or unfamiliarity with the technology. Additionally, integration with legacy databases and systems often requires significant technical overhauls, which can be resource-intensive. Government regulations and bureaucratic approval processes can further delay or complicate adoption.

Another significant limitation lies in interoperability and standardization. There is currently no universally accepted standard for issuing and verifying blockchain-based academic credentials. This creates fragmentation and can hinder cross-border or inter-institutional recognition of certificates. Moreover, while blockchain enhances transparency, storing even hashed or encrypted personal data on-chain raises privacy concerns under data protection laws like GDPR. The immutability of blockchain, though a strength, can also become a drawback if errors need to be corrected or records updated. From the user's perspective, managing cryptographic keys or digital wallets may introduce usability challenges, particularly for students or staff unfamiliar with blockchain systems. Lastly, though BlockVerify focuses on energy-efficient blockchains, sustainability remains a concern compared to conventional databases, especially at scale.

8.3 Future Enhancements

In the future, BlockVerify can evolve into a more comprehensive digital credentialing ecosystem by incorporating Decentralized Identity (DID) standards and Verifiable Credentials (VCs) as defined by the W3C. This would allow users to manage not just certificates but their entire digital identity in a secure, interoperable format. By enabling users to store and selectively share personal data, academic records, and qualifications, the platform can offer a more holistic solution for identity verification. The addition of cross-chain interoperability can further enhance the system, allowing certificates to be verified across multiple blockchain networks, which would increase flexibility and future-proof the platform.

User experience is another area for enhancement. Developing intuitive mobile applications, integrating biometric authentication, and offering offline access options can improve accessibility, especially for users in low-bandwidth or rural environments. Furthermore, incorporating multi-language support and accessibility features for individuals with disabilities will make the platform more inclusive.

From a security and privacy perspective, implementing zero-knowledge proofs (ZKPs) will allow verification without revealing sensitive data, greatly enhancing user privacy. To support a growing global user base, the platform can also leverage cloud-native infrastructure for scalability and high availability. Finally, expanding into micro-credentialing, professional licenses, and skill badges would position BlockVerify as a versatile solution for lifelong learning and career development in the digital age.

Chapter 9

SUSTAINABLE DEVELOPMENT GOALS (SDGs)

9.1 Alignment with SDGs

The BlockVerify project aligns with several United Nations Sustainable Development Goals (SDGs), including:

- **SDG 4: Quality Education** – BlockVerify directly supports inclusive and equitable quality education by enabling secure, tamper-proof digital certification. It helps combat credential fraud, improves trust in academic achievements, and enhances access to verified learning records for students, especially in underserved regions.
- **SDG 9: Industry, Innovation, and Infrastructure** – By integrating blockchain technology into academic credentialing, the project promotes innovation and strengthens infrastructure in the education sector. It demonstrates the application of cutting-edge decentralized technologies to solve real-world problems in document verification and institutional trust.
- **SDG 16: Peace, Justice, and Strong Institutions** – BlockVerify fosters transparency and accountability by ensuring the authenticity of certificates through immutable blockchain records. This contributes to building strong, trustworthy institutions and reduces the risk of corruption in credential issuance and recognition.
- **SDG 10: Reduced Inequalities** – Through a decentralized and accessible verification platform, BlockVerify can help bridge the gap between institutions of varying resources, enabling equal recognition of qualifications regardless of geographic or economic context.

9.2 Relevance of the Project to Specific SDG

- **SDG 4: Quality Education** – BlockVerify addresses a critical challenge in global education: the verification and authenticity of academic credentials. By issuing certificates on a blockchain, the project ensures that educational qualifications are tamper-proof, instantly verifiable, and accessible across borders. This fosters trust in digital learning systems and facilitates lifelong learning by making records portable and transparent. It is especially impactful in regions with weak institutional infrastructure or a high prevalence of credential fraud.
- **SDG 9: Industry, Innovation, and Infrastructure** – BlockVerify embodies technological innovation by utilizing blockchain and smart contracts to create a secure, decentralized infrastructure for credential issuance and verification. This reduces dependency on centralized authorities and enhances the scalability, reliability, and efficiency of verification processes in academic and professional environments. The platform represents a modern, infrastructure-oriented solution to a widespread problem in education and workforce development.

9.3 Potential Social and Environmental Impact

Social Impact:

BlockVerify significantly contributes to educational equity and digital inclusion by offering a robust and transparent platform for managing academic credentials. In many regions, especially underserved and remote communities, students face substantial challenges in proving their educational achievements due to a lack of infrastructure, inconsistent record-keeping, or the loss of physical documents. BlockVerify addresses these issues by enabling secure, tamper-proof digital certificates that can be accessed and shared globally. This ensures that learners are no longer restricted by geography, institutional inefficiencies, or administrative hurdles. The platform promotes self-sovereign identity, giving individuals complete ownership and control over their credentials. This autonomy reduces dependency on centralized authorities and intermediaries, thereby enhancing trust and personal agency. Furthermore, by streamlining the verification process for employers and academic institutions, BlockVerify accelerates access to job markets and educational opportunities, ultimately fostering greater social mobility and economic empowerment.

Environmental Impact:

BlockVerify contributes to environmental sustainability by eliminating the traditional reliance on paper-based documentation and postal services. The transition to a fully digital certification process reduces the consumption of natural resources, such as paper and ink, and significantly cuts down on carbon emissions associated with printing, mailing, and physical storage. Additionally, the platform is designed to operate using energy-efficient blockchain protocols like Proof-of-Authority (PoA) or other low-energy consensus mechanisms. These methods require significantly less computational power compared to more energy-intensive models like Proof-of-Work (PoW), thus ensuring that the system maintains a minimal environmental footprint. By combining technological innovation with eco-conscious design, BlockVerify supports the development of sustainable digital infrastructure. This approach aligns with broader global efforts to reduce environmental impact and encourages other educational and administrative institutions to adopt greener digital alternatives.

Chapter 10

PLAGIARISM REPORT

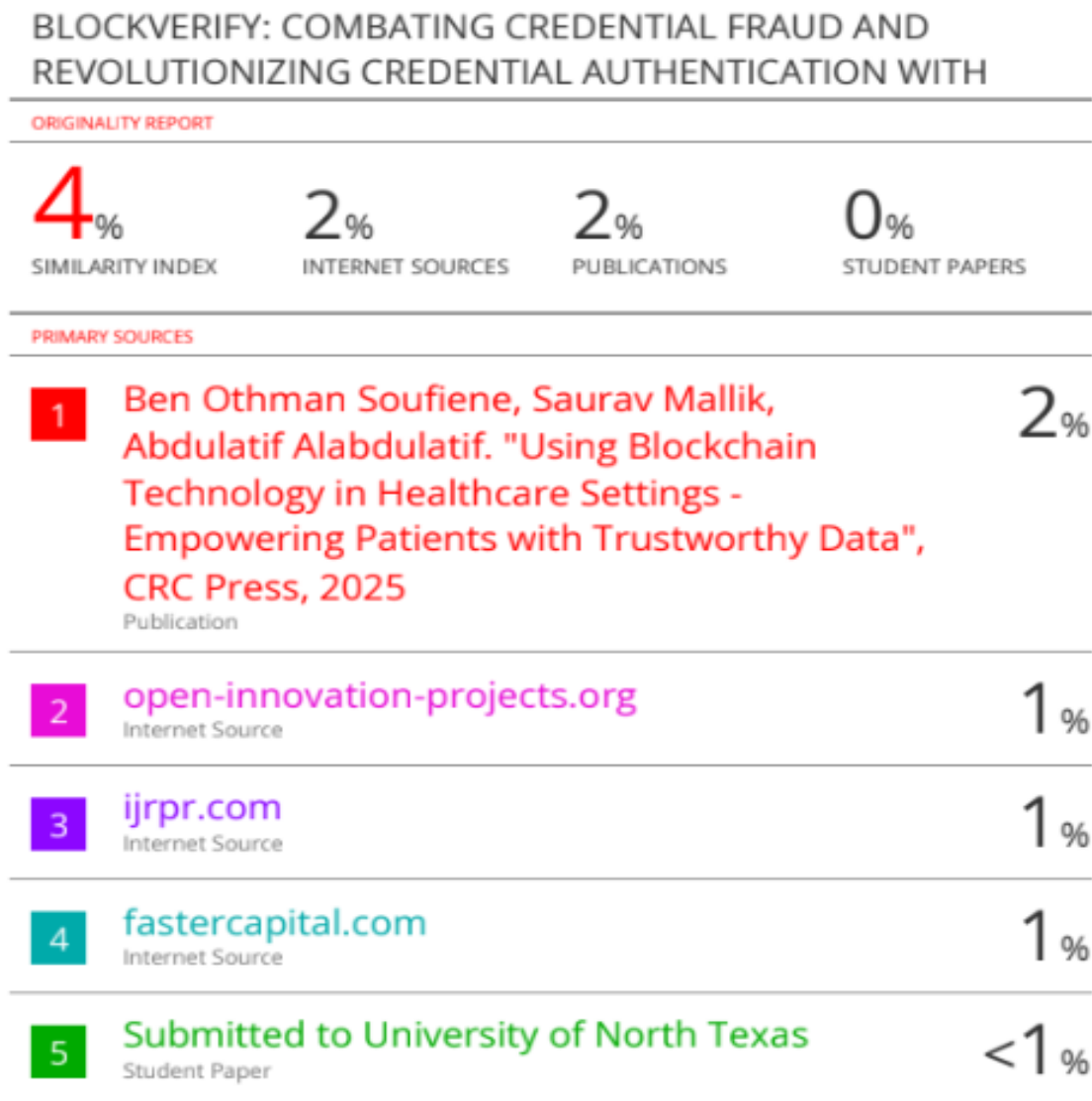


Figure 10.1: Plagiarism Report of Certificate generation using blockchain

Chapter 11

SOURCE CODE

11.1 Source Code

```
1 "use client";
2 import { useEffect, useState } from "react";
3 import { Trash2, MoveLeft } from "lucide-react";
4 import { useParams, useRouter } from "next/navigation";
5 import axios from "axios";
6 import { generateRandom } from "@/utils/randomId";
7 import { addCertificate } from "@/app/api/contract/deploy";
8 import { useAuthState } from "react-firebase-hooks/auth";
9 import { auth } from "@/config/firebase";
10
11 interface Participant {
12   pId: string;
13   pName: string;
14   pEmail: string;
15   phone: string;
16 }
17
18 type Event = {
19   _id: string;
20   eventId: string | undefined;
21   eventName: string;
22   location: string;
23   description: string;
24   certificateTemplate: string | null;
25   date: Date | undefined;
26   isIssued: boolean;
27 };
28
29 export default function Page() {
30   const [user] = useAuthState(auth);
31   const [participants, setParticipants] = useState<Participant[]>([]);
32   const [name, setName] = useState("");
33   const [email, setEmail] = useState("");
34   const [phone, setPhone] = useState("");
35   const { id } = useParams();
36   const [event, setEvent] = useState<Event | null>(null);
37   const [loading, setLoading] = useState(false); // Loading state for API calls
38   const router = useRouter();
```

```

39  const [hashes, setHashes] = useState([])
40  const [contractAddress, setContractAddress] = useState(null)
41
42  const fetchData = async () => {
43    if (!id) return;
44    setLoading(true);
45    try {
46      const [eventResponse, participantsResponse] = await Promise.all([
47        axios.get(`/api/events/${id}`),
48        axios.get(`/api/participants?eventId=${id}`),
49      ]);
50      setEvent(eventResponse.data);
51      setParticipants(participantsResponse.data.participants || []);
52    } catch (error) {
53      console.error("Error fetching data:", error);
54    } finally {
55      setLoading(false);
56    }
57  };
58  const checkOrganizationExists = async () => {
59    if (user) {
60      try {
61        const response = await axios.get(`/api/organization?orgId=${user.uid}`);
62        setContractAddress(response.data?.contractAddress)
63      } catch (error) {
64        console.error("Error checking organization:", error);
65      }
66    }
67  };
68
69
70
71  const handleSubmit = async (e: React.FormEvent) => {
72    e.preventDefault();
73    if (!name || !email || !phone) return;
74    setLoading(true);
75    try {
76      await axios.post("/api/participants", {
77        pId: generateRandom(),
78        pName: name,
79        pEmail: email,
80        phone,
81        eventId: id,
82      });
83      setName("");
84      setEmail("");
85      setPhone("");
86      fetchData(); // Refresh data after successful submission
87    } catch (error) {
88      console.error("Error adding participant:", error);

```

```

89     } finally {
90         setLoading(false);
91     }
92 };
93
94 const deleteParticipant = async (participantId: string) => {
95     setLoading(true);
96     try {
97         await axios.delete("/api/participants", {
98             data: { participantId },
99         });
100         fetchData(); // Refresh data after deletion
101     } catch (error) {
102         console.error("Error deleting participant:", error);
103     } finally {
104         setLoading(false);
105     }
106 };
107
108 const issueCertificates = async () => {
109     if (!id) return;
110     setLoading(true);
111     try {
112
113         await axios.post("/api/certificates", { eventId: id }).then((response) => {
114             setHashes(response.data)
115         })
116         const vaild = await addCertificate(contractAddress, hashes)
117         await axios.put('/api/certificates', {
118             eventId: event?.id,
119             blockHash: vaild
120         })
121         console.log(vaild)
122         fetchData(); // Refresh data after issuing certificates
123     } catch (error) {
124         console.error("Error issuing certificates:", error);
125     } finally {
126         setLoading(false);
127     }
128 };
129
130 useEffect(() => {
131     fetchData(); // Fetch data on component load
132     checkOrganizationExists()
133 }, []);
134
135 return (
136     <div className="p-4 m-8 flex flex-col gap-6">
137         <div
138             onClick={() => router.push("/dash")}

```

```

139     className="fixed top-4 left-12 flex gap-2 cursor-pointer hover:underline"
140   >
141     <MoveLeft /> Home
142   </div>
143   { /* Event Details */ }
144   <div className="shadow-md border rounded-md p-6">
145     <h2 className="text-xl font-bold">{event?.eventName}</h2>
146     <p className="text-gray-600 mt-2">{event?.description}</p>
147     <div className="mt-4 space-y-1">
148       <p>
149         <strong>Date:</strong> {event?.date?.toString()}
150       </p>
151       <div className="flex justify-between">
152         <p>
153           <strong>Location:</strong> {event?.location}
154         </p>
155         <div className="flex text-lg">
156           Status:{ " " }
157           {event?.isIssued ? (
158             <p className="font-semibold text-customGreen">Already Issued </p>
159           ) : (
160             <p className="font-semibold text-gray-300">Not Issued </p>
161           )}
162         </div>
163       </div>
164     </div>
165   </div>
166
167   <div className="flex flex-col md:flex-row gap-8">
168     { /* Add Participant */ }
169     { !event?.isIssued && (
170       <div className="shadow-md border rounded-md p-6 w-full md:w-1/2">
171         <h3 className="text-lg font-bold mb-4">Add Participant </h3>
172         <form className="space-y-4" onSubmit={handleSubmit}>
173           <div>
174             <label
175               htmlFor="name"
176               className="block text-sm font-medium text-gray-300"
177             >
178               Name
179             </label>
180             <input
181               id="name"
182               type="text"
183               value={name}
184               onChange={(e) => setName(e.target.value)}
185               className="w-full p-3 border bg-transparent border-gray-300 rounded-md focus:
186                 outline-none focus:ring-2 focus:ring-blue-500"
187               required
188             />

```

```

188     </div>
189   <div>
190     <label
191       htmlFor="email"
192       className="block text-sm font-medium text-gray-300"
193     >
194       Email
195     </label>
196     <input
197       id="email"
198       type="email"
199       value={email}
200       onChange={(e) => setEmail(e.target.value)}
201       className="w-full p-3 border bg-transparent border-gray-300 rounded-md focus:
202         outline-none focus:ring-2 focus:ring-blue-500"
203       required
204     />
205   </div>
206   <div>
207     <label
208       htmlFor="phone"
209       className="block text-sm font-medium text-gray-300"
210     >
211       Phone No.
212     </label>
213     <input
214       id="phone"
215       type="text"
216       value={phone}
217       onChange={(e) => setPhone(e.target.value)}
218       className="w-full p-3 border border-gray-300 bg-transparent rounded-md focus:
219         outline-none focus:ring-2 focus:ring-blue-500"
220       required
221     />
222   </div>
223   <button
224     type="submit"
225     className={`w-full py-2 px-4 rounded-md ${
226       loading
227       ? "bg-gray-300 text-gray-600 cursor-not-allowed"
228       : "bg-indigo-600 text-white hover:bg-indigo-700 focus:outline-none focus:ring-2
229         focus:ring-indigo-500"
230     }`}
231     disabled={loading}
232   >
233     {loading ? "Adding..." : "Add Participant"}
234   </button>
235 </form>
236 </div>
237 )}

```

```

235
236 { /* Participants */}
237 <div
238   className={ `shadow-md border rounded-md p-6 ${
239     !event?.isIssued
240       ? "max-h-96 w-full md:w-1/2 overflow-y-scroll"
241       : "min-h-fit min-w-full px-32"
242     } ` }
243 >
244   <h3 className="text-lg font-bold mb-4 text-customGreen">Participants </h3>
245   { participants.length === 0 ? (
246     <p className="text-gray-600">No participants yet.</p>
247   ) : (
248     <ul className="space-y-4">
249       { participants.map((participant) => (
250         <li
251           key={ participant.pId }
252           className="flex justify-between items-center border-b pb-2"
253         >
254           <span>
255             { participant.pName } ( { participant.pEmail }, { " " }
256             { participant.phone } )
257           </span>
258           { !event?.isIssued && (
259             <button
260               className="text-red-600 hover:text-red-800 focus:outline-none"
261               onClick={() => deleteParticipant(participant.pId)}
262               disabled={ loading }
263             >
264               <Trash2 className="w-5 h-5" />
265             </button>
266           ) }
267         </li>
268       ) ) }
269     </ul>
270   ) }
271 </div>
272 </div>
273
274 { /* Issue Certificates */}
275 { !event?.isIssued && (
276   <button
277     onClick={ issueCertificates }
278     disabled={ participants.length === 0 || loading }
279     className={ `py-2 w-full px-4 rounded-md ${
280       participants.length === 0 || loading
281         ? "bg-gray-300 text-gray-600 cursor-not-allowed"
282         : "bg-green-600 text-white hover:bg-green-700 focus:outline-none focus:ring-2 focus:
283           ring-green-500"

```



```
284         >
285         {loading ? "Issuing..." : "Issue Certificates"}
286     </button>
287     )}
288 </div>
289 );
290 }
```


References

- [1] A. Grech and A. F. Camilleri (2017). Blockchain in education: An introduction, Joint Research Centre (JRC), European Commission, pp. 1–44.
- [2] B. K. Mohanty, R. Sharma, and A. P. Mishra (2021). Comparative analysis of blockchain-based certification platforms: Consensus mechanisms and scalability challenges, *IEEE Transactions on Learning Technologies*, 14(3), pp. 324–337.
- [3] C. Rong, X. Zhang, and J. Wu (2021). Smart contract security in blockchain-based credentialing applications: A survey of vulnerabilities and mitigation techniques, *IEEE Transactions on Software Engineering*, 47(5), pp. 987–1001.
- [4] D. Patel and A. Shah (2022). Cryptographic hashing and blockchain for securing academic credentials: A detailed analysis, *IEEE Transactions on Information Forensics and Security*, 17, pp. 3421–3435.
- [5] E. Liu, Z. Chen, and L. Wang (2022). Enterprise adoption of blockchain-based credential verification: Challenges and opportunities. In *Proc. IEEE Int. Conf. Business Inf. Manage. (BIM)*, pp. 45–51.
- [6] J. A. Halderman, D. M. Levin, and B. Waters (2018). Security analysis of blockchain-based credential verification systems, *IEEE Transactions on Dependable and Secure Computing*, 15(6), pp. 1050–1063.
- [7] K. Singh and P. Verma (2023). Zero-knowledge proofs for privacy-preserving blockchain-based credential verification, *IEEE Transactions on Emerging Topics in Computing*, 11(1), pp. 210–225.
- [8] M. Sharples and J. Domingue (2016). The blockchain for open learning records and lifelong learning, *Proceedings of the European Conference on Technology Enhanced Learning (ECTEL)*, pp. 311–316.
- [9] S. Kim and Y. Kang (2019). Decentralized identity management using blockchain: A self-sovereign approach, *IEEE Access*, 7, pp. 179174–179188.
- [10] T. Nguyen, H. Do, and K. Tran (2020). Blockchain adoption in higher education: Issuing digital diplomas with security and efficiency, *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 1–6.



**4TH INTERNATIONAL CONFERENCE ON INFORMATION TECHNOLOGY,
CIVIL INNOVATION, SCIENCE, AND MANAGEMENT (ICITSM'25)**

CERTIFICATE

of Presentation on 28 & 29 April 2025

This Certificate is Proudly Presented To

LIJETHA C JAFFRIN

**BLOCKVERIFY: COMBATING CREDENTIAL FRAUD AND REVOLUTIONIZING CREDENTIAL
AUTHENTICATION WITH BLOCKCHAIN TECHNOLOGY**

in Conference of Proceedings of the **4th International Conference on Information
Technology, Civil Innovation, Science, and Management (ICITSM'25)** Organized by
Department of Civil Engineering at K.S.R. College of Engineering, Namakkal,
Tamil Nadu, India

Dr S Senthilkumar,
Professor and Head,
Department of Civil Engineering,
K. S. R. College of Engineering,
Tiruchengode



Ms. Gugapriya Sivakumar
Industry Partner,
NTL Technology, Erode.



**4TH INTERNATIONAL CONFERENCE ON INFORMATION TECHNOLOGY,
CIVIL INNOVATION, SCIENCE, AND MANAGEMENT (ICITSM'25)**

CERTIFICATE

of Presentation on 28 & 29 April 2025

This Certificate is Proudly Presented To

UNDAVALLI VARSHITHA

**BLOCKVERIFY: COMBATING CREDENTIAL FRAUD AND REVOLUTIONIZING CREDENTIAL
AUTHENTICATION WITH BLOCKCHAIN TECHNOLOGY**

in Conference of Proceedings of the **4th International Conference on Information
Technology, Civil Innovation, Science, and Management (ICITSM'25)** Organized by
Department of Civil Engineering at K.S.R. College of Engineering, Namakkal,
Tamil Nadu, India

Dr S Senthilkumar,
Professor and Head,
Department of Civil Engineering,
K. S. R. College of Engineering,
Tiruchengode



Ms. Gugapriya Sivakumar
Industry Partner,
NTL Technology, Erode.

**4th International Conference on Information Technology, Civil
Innovation, Science, and Management (ICITSM'25)**

28 & 29 April 2025

K.S.R. College of Engineering, Namakkal, Tamil Nadu, India



PAYMENT INVOICE

Receipt Number : 2025 - ICITSM - 430

Received From : LJETHA C JAFFRIN, UNDAVALLI VARSHITHA

Amount Paid : 12,000

Amount in Words : Twelve Thousand Only

Paper Title : BLOCKVERIFY: COMBATING CREDENTIAL
FRAUD AND REVOLUTIONIZING
CREDENTIAL AUTHENTICATION WITH
BLOCKCHAIN TECHNOLOGY


Dr S Senthilkumar,
Professor and Head,
Department of Civil Engineering,
K. S. R. College of Engineering,
Tiruchengode

